World Models: Computing the Uncomputable

A Co-Written Essay with Pim De Witte

“I wanted to fall asleep last night. Instead, I started imagining all of the scenarios I might run into the next day, and how I might react to them.”

This is a common experience. As humans, we imagine easily, whether it's complex sports stadiums, potential romance, or heated discussions. We don't have to work harder to imagine ourselves at the next Manchester United game than we do to imagine talking to a friend we've known for years, even though imagining a Manchester game includes simulating and modeling the behavior of thousands of people, something that would take years for traditional computers and game engines today[1].

Think about writing the code to describe the Man U match: at any moment, a fan might bring a random, home-crafted flag. The entire stadium starts singing a song related to it. Only some will sing, though; others will jump with their kids, while an old couple sits still, wondering if this is their last game together, soaking in every second in silence.

The world is a place where unexpected futures unfold, but in somewhat predictable ways. As humans, we can envision almost all of them with roughly the same amount of effort with a very similar amount of time given to each thought. Computers can't.

It's no wonder traditional computing struggles with this complexity. Imagine anticipating and coding each and every action, as well as the interactions between all of those actions. Mathematically, in a traditional engine, simulating *N* fans is at least an *O(N)* or *O(N2)* problem. Each person, flag, chair, and ball must be explicitly calculated — and really, the interactions between them need to be calculated, too.

In robotics, machines must respond to situations in the real world in the same amount of time, regardless of their complexity, even though, in traditional computing, different situations can take wildly different amounts of time to simulate. This has been a major bottleneck for robotics and embodied AI progress.

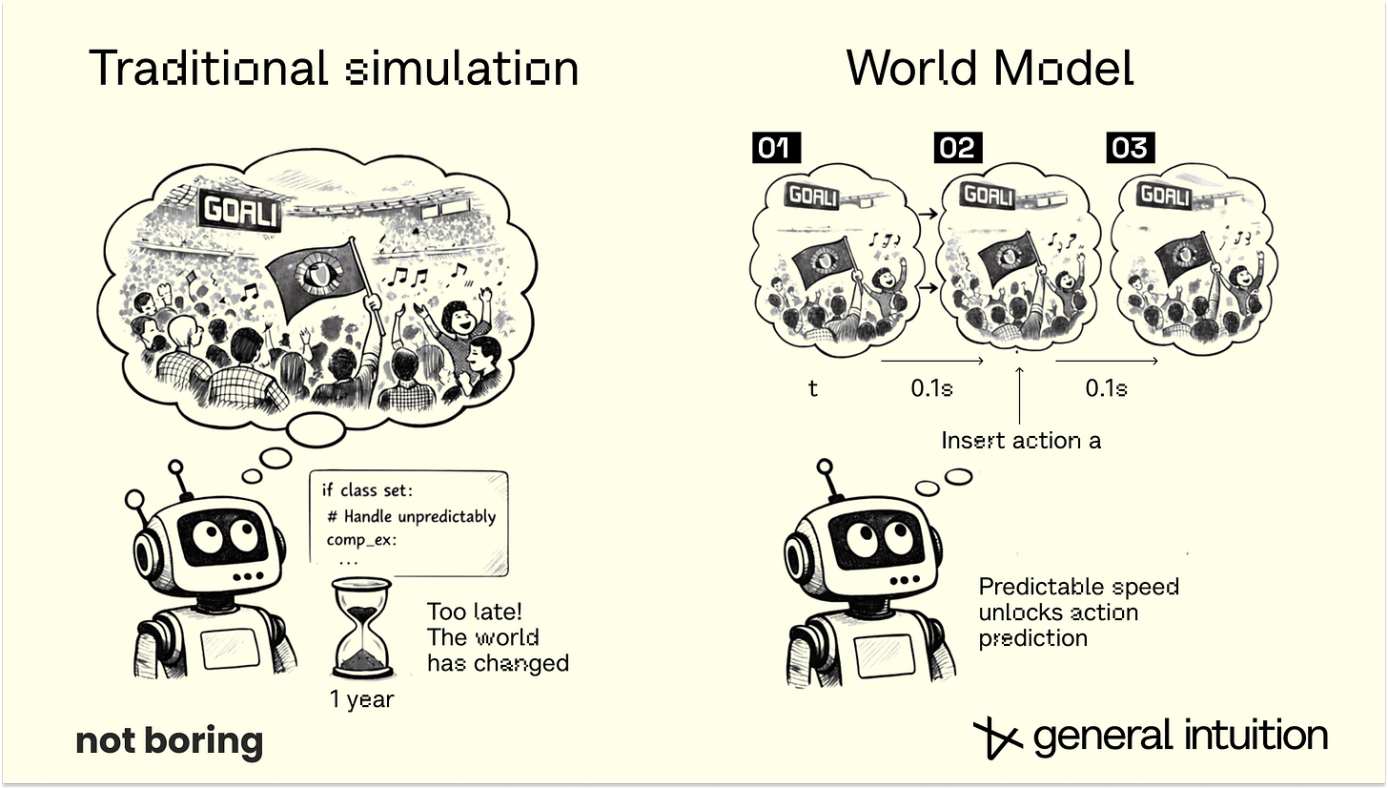
World Models are a solution to that problem.

World Models learn to predict those dynamics from video and, often, the actions taken in them. They reduce situations that are dynamic and computationally difficult to simulate at scale — including stochastic, action-dependent group behavior like soccer games — into a single fixed cost operation in a neural network.

In a World Model, the *entire stadium* is simulated as a fixed cost forward pass through the neural network. The complexity of the scene doesn't exponentially slow down the 'engine' during inference because the weights have already absorbed the patterns of the world in training.

How? **Actions.**

Actions act as a form of compression to predict unfolding dynamics: they hold the information to unroll future states in an environment, until more actions take place and add new inputs into the environment. Each action carries enough information to predict what happens next, until the next action updates the picture.



This **action-conditioned** approach allows models to learn and plan interactively. Today, this is intractable in even the best simulation engines, and definitely not at predictable compute costs. Actions help models interact with the world like we do.

Over and over again, every single day, you observe, you compute, you decide what to do, you act. This is life. At any point, all gathered information about space and time collapses into the action you take.

For computers, actions are a cheat code around the costs of simulation. If human brains are much more efficient than best-in-class LLMs, then we can get all of that computation practically for free by observing how humans respond to the countless variables in their environments. This gives us a way to do non-deterministic computing efficiently and create simulations that shouldn't be possible under traditional compute constraints.

This ability to **compute the uncomputable** is why we believe World Models will unlock progress in embodied AI in a way that current model architectures can't.

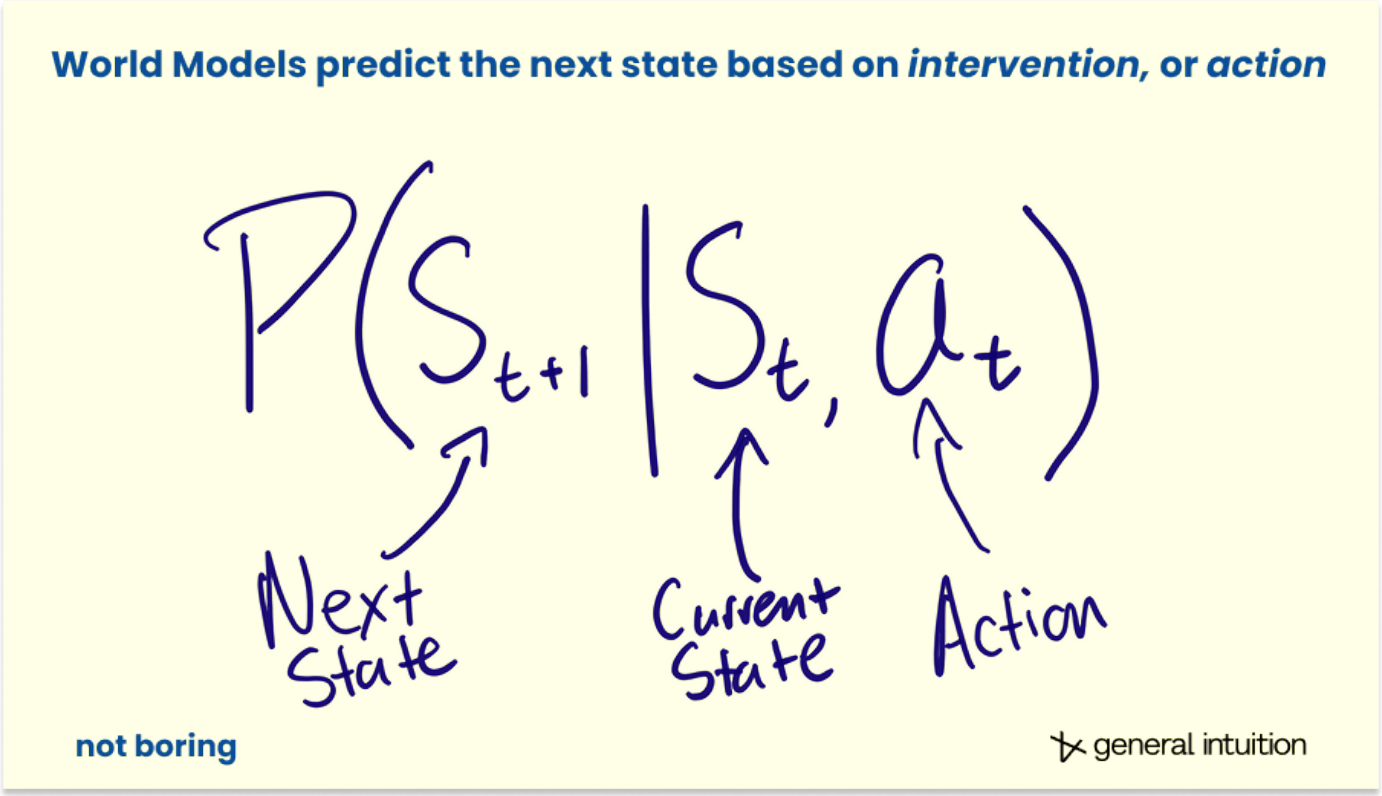
Think about models like dreams.

Have you ever had a dream where you simply stood and watched what was happening without the ability to intervene? **That's a video model**.

The real world is different. It responds to what you do or instruct to do, and predicts the full range of things that could happen as a result, not just the single most likely or most entertaining next frame.

Have you ever had a lucid dream in which you were able to shape the story inside the mind-generated dreamscape? **That's a World Model**.

More formally, while a standard video model predicts the next frame based on probability, P(xt+1 | xt), **a World Model predicts the next state based on *intervention*, P(st+1 | st, at).**



That **at**, the action at time t, is the magic.

At **General Intuition**, we believe (and are seeing early signs) that World Models are a new and potentially more powerful class of foundation model than LLMs for environments that require deep spatial and temporal reasoning. Environments like our real world.

World models — these systems that learn from watching the world and the actions taken in it — are a fundamentally new kind of foundation model. They can compute what was previously uncomputable.

They will matter far more than anyone currently realizes, because they offer a path to general intelligence that language and code alone cannot. Being human, after all, is spending a lifetime **taking actions based on what we experience, observe, and learn**.

Pause. You might be confused by that claim, that World Models offer a path to general intelligence that LLMs cannot. Understandably so.

World Models are getting a lot of attention as of late. Yann LeCun, who has been skeptical that LLMs are the path to general intelligence, just announced that he raised $1.03 billion for AMI. Fei-Fei Li's World Labs has also raised more than $1 billion to pursue World Models. Google DeepMind, which has the closest thing to an infinite money printer in tech, is betting money on World Models too. But what we've seen so far from that investment are cool videos and 3D worlds.

LLMs can quote Shakespeare and solve Erdős Problems. World Models, on the other hand, still seem more like a path to the Metaverse than a path to general intelligence.

But part of the reason World Models don't yet have the hype of LLMs is that their *definitions* are still shaky.

What are World Models? We've already said that video models don't fit the definition. 3D space models don't, either. That said, both may be paths to World Models. Are the models that animate robots today World Models? Not really, although some are, and even the ones that aren't share features with World Model architectures.

As always, hype adds to confusion. "My prediction is that 'World Models' will be the next buzzword," Alexandre LeBrun, the CEO of AMI Labs (which is definitely a World Model company) told TechCrunch. "In six months, every company will call itself a World Model to raise funding."

Hype is a small part of it. **What we — and everyone else building in this space — believe is that World Models are the path to controlling machines in the physical world.** There are differences in what we believe this path will look like. But all of us believe that the future runs through World Models.

"...very few understand how far-reaching this shift is...," NVIDIA Director of Robotics and Distinguished Scientist Jim Fan said recently. "Unfortunately, the most hyped use case of World Models right now is AI video slop (and coming up, game slop). I bet with full confidence that 2026 will mark the first year that Large World Models lay real foundations for robotics, and for multimodal AI more broadly."

Today, we'd like to welcome you into the group of the "very few" who "understand how far-reaching this shift is." We are going to share the history of World Models, the state of the field as it stands today, broad explanations of the approaches each major lab is taking, and the convictions that drive General Intuition's directions.

Whether you come with us is up to you. You take the blue pill, the story ends. You wake up in your bed and believe whatever you want to believe. You take the red pill... you stay in Wonderland, and we show you how deep the rabbit hole goes.

For example.... how can you be sure that you're not an Agent operating inside of a World Model yourself?

Can Agents Learn Inside of Their Own Dreams?

Wake up, Neo.

World models aren't a new idea. They are one of our oldest. Since humans gained the ability to think about our place in the universe, to ask why we are here, we have pondered whether our reality is just a simulation.

In 380 BC, Plato, via Socrates, offered *The Allegory of the Cave*. Imagine human beings who live underground in a cave, necks chained, forced to look ahead at the shadows on the wall. Those humans would believe those shadows to *be* reality, when in fact they are mere shadows of reality. This was Plato's metaphor. He suggests that we are all stuck in the cave, necks chained, mistaking our perception for true reality.

Eighty years later, Chinese Daoist philosopher Zhuangzi contemplated similar questions in a passage of his *Butterfly Dream*:

> *Once Zhuang Zhou dreamt he was a butterfly, a butterfly flitting and fluttering around, happy with himself and doing as he pleased. He didn't know he was Zhuang Zhou. Suddenly, he woke up and there he was, solid and unmistakable Zhuang Zhou. But he didn't know if he was Zhuang Zhou who had dreamt he was a butterfly, or a butterfly dreaming he was Zhuang Zhou. Between Zhuang Zhou and a butterfly there must be some distinction! This is called the Transformation of Things.*

As the centuries passed and our technological capabilities evolved, sci-fi writers joined the long lineage of thinkers inquiring about the true nature of reality. Frederik Pohl's 1955 *The Tunnel Under the World*. Daniel F. Galouye's *Simulacron-3*. Stanislaw Lem's *Non Serviam*. Vernor Vinge's *True Names*. William Gibson's *Neuromancer*. Neal Stephenson's *Snow Crash*. All painted textual pictures of simulated worlds.

During a 1977 speech in Metz, France, sci-fi legend Philip K. Dick confidently told the audience: "We are living in a computer-programmed reality, and the only clue we have to it is when some variable is changed2, and some alteration in our reality occurs."

Your first interaction with the simulation was probably *The Matrix.* Ours was. In the original script for *The Matrix,* the Wachowskis conceived of the Matrix as a simulation collectively produced by human brains chained into a neural network.

Ignorance is Bliss

The studio thought humans-as-computers was too confusing a concept for mass-market audiences, so they made the thermodynamically questionable decision to turn humans into batteries that powered the simulation. That was probably the right commercial call. The Matrix franchise has done nearly $2 billion in worldwide gross. More impactfully, it introduced the masses to the idea of a simulated world generated indistinguishable from the "real" one.

It's no wonder that this idea has taken hold of our collective imagination. It's certainly the right kind of weird but it's also surprisingly hard to disprove. **If the observations are the same, and the actions are the same, then the computation is the same.** If what you see is the same and what you do is the same, it doesn't matter whether you're in a simulation or reality. It doesn't matter whether you're walking down a real street or a simulated one. Your brain processes both identically. Neo had no idea he was in the Matrix until Morpheus woke him up.

Christopher Nolan, throwing audience confusion to the wind — savoring it, even — released *Inception*[3] in 2010. Dreams within dreams within dreams.

Nolan's central premise is that the dream is a controllable space from which information can be extracted or, more importantly, into which information can be implanted.

But it's all just sci-fi, right?

In 1990, Jürgen Schmidhuber, a young researcher at the Technical University of Munich, published *Making the World Differentiable*.

The paper proposed building a **recurrent neural network (RNN),** a neural network with two jobs: first, learn to predict what happens next in a simulated world and second, use that simulated world to train an Agent to act in it.

The Agent wouldn't need to interact with a "real" environment at all. It could learn inside the model. Inside a dream.

Making the World Differentiable: On Using Self-Supervised Fully Recurrent Neural Networks for Dynamic Reinforcement Learning and Planning in Non-Stationary Environments

Jürgen Schmidhuber*
Institut für Informatik
Technische Universität München
Arcisstr. 21, 8000 München 2, Germany
schmidhu@tumult.informatik.tu-muenchen.de

Jürgen Schmidhuber

The following year, Richard Sutton, of *Bitter Lesson* fame, dreamt up a similar idea. In *Dyna, an Integrated Architecture for Learning, Planning, and Reacting*, he argued that learning, planning, and reacting shouldn't be separate systems. They should be unified in a single architecture. Which would mean that it's technically possible to build a model of the world, practice inside it, and transfer what you learn back to reality.

Both papers were visionary. They would have a lasting impact as progress in the field enabled the researchers' visions to become reality. But coming when they did, both papers may as well have been sci-fi.

In 1990, the world had something like 100 trillion to 1 quadrillion times less compute than we have today. Back then, the entire world had maybe 10-100 gigaFLOPS of total capacity. Tens of zettaflops (10^22 FLOPS) of computing power were sold in 2024 alone. In 1990, the global digital datasphere was approximately 10 petabytes, a volume so small it could barely hold 0.005% of the video data we now use for a single training run. By 2026, that volume has exploded by a factor of 22 million to 221 zettabytes.

But technology improves, and the most powerful dreams do not die.

Nearly three decades later, in March 2018, David Ha (then at Google Brain) and Schmidhuber published a paper titled *World Models*.[4]

World Models

David Ha [1] Jürgen Schmidhuber [2 3]

Abstract

We explore building generative neural network models of popular reinforcement learning environments. Our *world model* can be trained quickly in an unsupervised manner to learn a compressed spatial and temporal representation of the environment. By using features extracted from the world model as inputs to an agent, we can train a very compact and simple policy that can solve the required task. We can even train our agent entirely inside of its own hallucinated dream generated by its world model, and transfer this policy back into the actual environment.

An interactive version of this paper is available at https://worldmodels.github.io

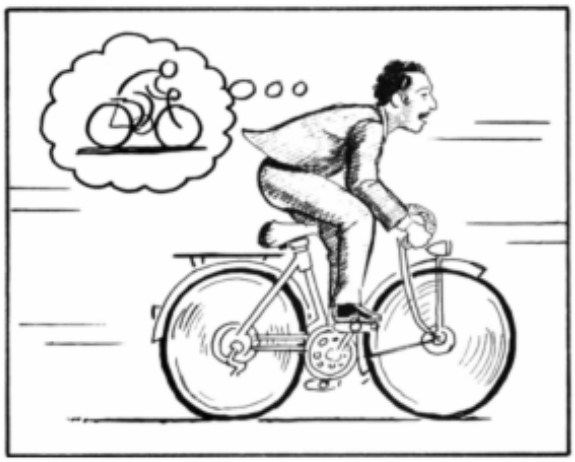
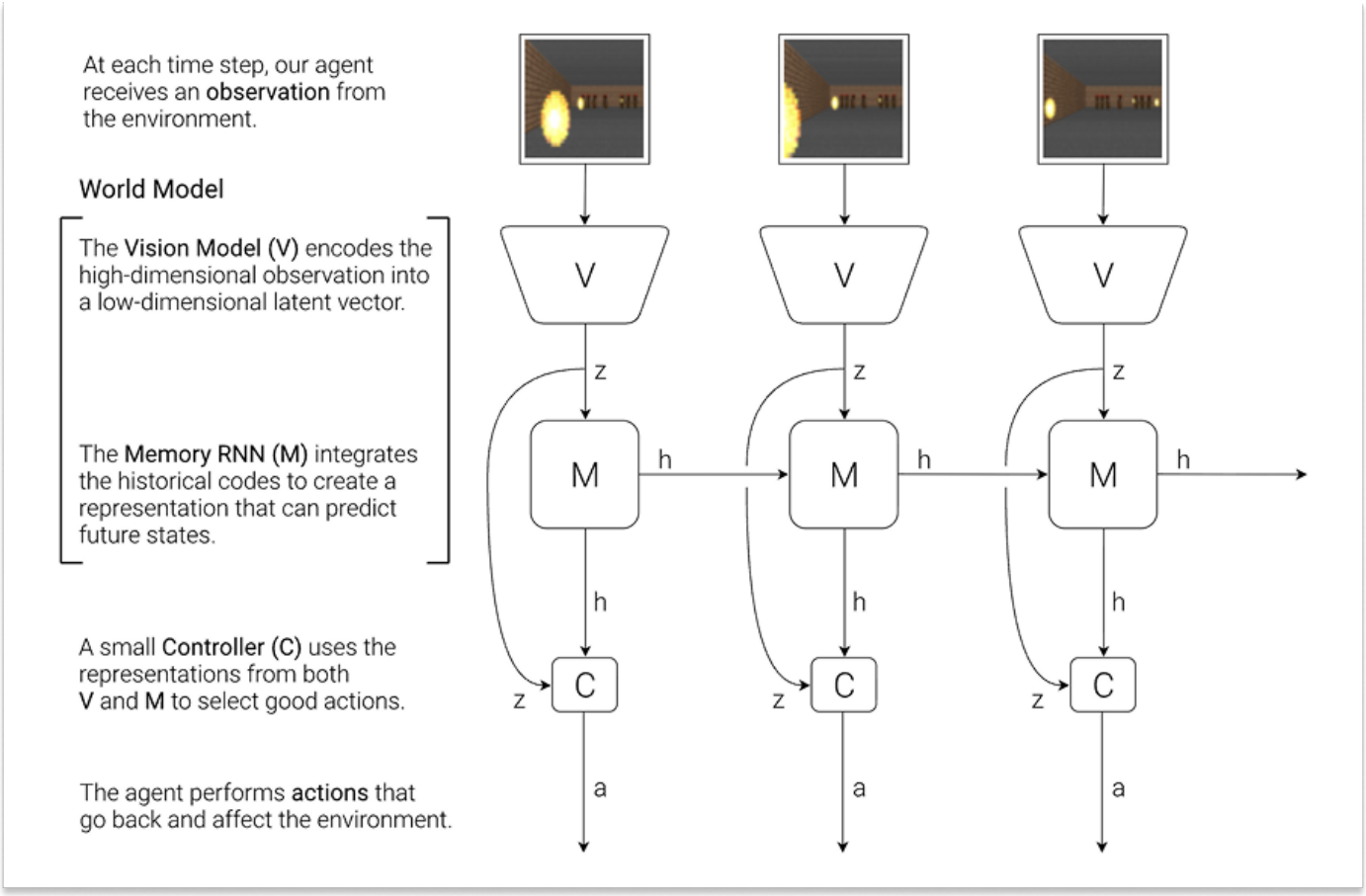


Figure 1. A World Model, from Scott McCloud's *Understanding Comics.* (McCloud, 1993; E, 2012)

The paper asked: ***Can agents learn inside of their own dreams?***

To answer their own question, Ha and Schmidhuber built a fictional system with three components: a **vision model (V)** that compressed raw pixel observations into a compact representation, a **memory model (M)**, a recurrent neural network that learned to predict what happens next, and a tiny **controller (C)** that decided what to do based only on V and M's outputs.

The **World Model** was V + M: it could take in observations and imagine plausible futures. The controller was the **Agent** or **policy**: it chose which actions to take.



World Model + Agent

The paper joined in conversation with those centuries of thought experiments, novels, and movies. A dream might be reality, reality might be dreams. But what if we could actually act in our dreams? What would that do to reality?

Ha and Schmidhuber trained their World Model on observations from a car racing game and a first-person shooter game. The World Model generated new digital worlds. Then, they let the Agent practice entirely inside the World Model's hallucinated dreams. Afterwards, they transferred the learned policy back to the actual environment.

And... it worked. The Agent could solve tasks it had never encountered in reality. The dream was real enough.

It was shocking, from a computer science perspective. But was it really so surprising? Isn't this how humans navigate the world?

Ha and Schmidhuber noted that humans constantly run World Models in their heads. A baseball player facing a 100 mph fastball has to decide how to swing before the visual signal of the ball's position even reaches their brain. The reason that every at-bat doesn't result in a strikeout is that batters don't react to reality, but to their brain's "internal World Model's" prediction of where the ball will be.

Donald Hoffman, Professor of Cognitive Sciences at University of California, Irvine, takes that idea a million steps further. He believes that we all walk around wearing "reality headsets" that simplify the staggering complexity of the quantum world into a user-friendly interface. Reality is too rich, so we navigate it via a sort of persistent waking dream.

This rabbit hole goes as deep as you want it to. But it's World Models all the way down.

Ha and Schmidhuber showed that computers might be able to approach the world like we do: creating simulations to predict future states based on actions, acting based on those predictions, updating, and looping.

Actions, not words.

Language is Not Enough (Neither is Code)

Let's play a game.

Clap your hands five times.

Now, instead of physically clapping your hands, I want you to *describe* clapping your hands using just words.

Where they are positioned in space, where they are relative to each other, by the picosecond. The points of contact. The sounds. What your hands look like as they move closer to each other, make contact, and pull apart. How they squish each other. What happens to the air between your two palms. What you see while your hands clap. Don't forget your arms. How do they bend to facilitate the claps? Remember to do this by the picosecond, too. How does the fabric on your sleeve respond? What is happening in the background? Did the person next to you notice you clapping? How did they respond? Did you get fired for clapping in the middle of the meeting, following the instructions of an essay you shouldn't have been reading while you should have been paying attention to work? Describe to me the vein on your boss' forehead. Is it popping?

You can't, can you? OK, stop. The point is made.

Language is an incredibly lossy compression of reality.

Language is important, of course. It is how we communicate and coordinate. The game Charades illustrates that to communicate ideas, language can be much more efficient than actions. LLMs are important in that capacity. But language alone is not enough.

What about code? Code is a form of very precise language that makes machines do things.

I asked Claude to "code me a simulation of hands clapping five times in a realistic environment." It built me this. Which looks very painful.

Hand-Clapping Simulation Generated by Claude

There is a belief that, with scale, language and code will be able to solve all spatial-temporal intelligence challenges and produce Artificial General Intelligence (AGI) or Artificial Superintelligence (ASI).

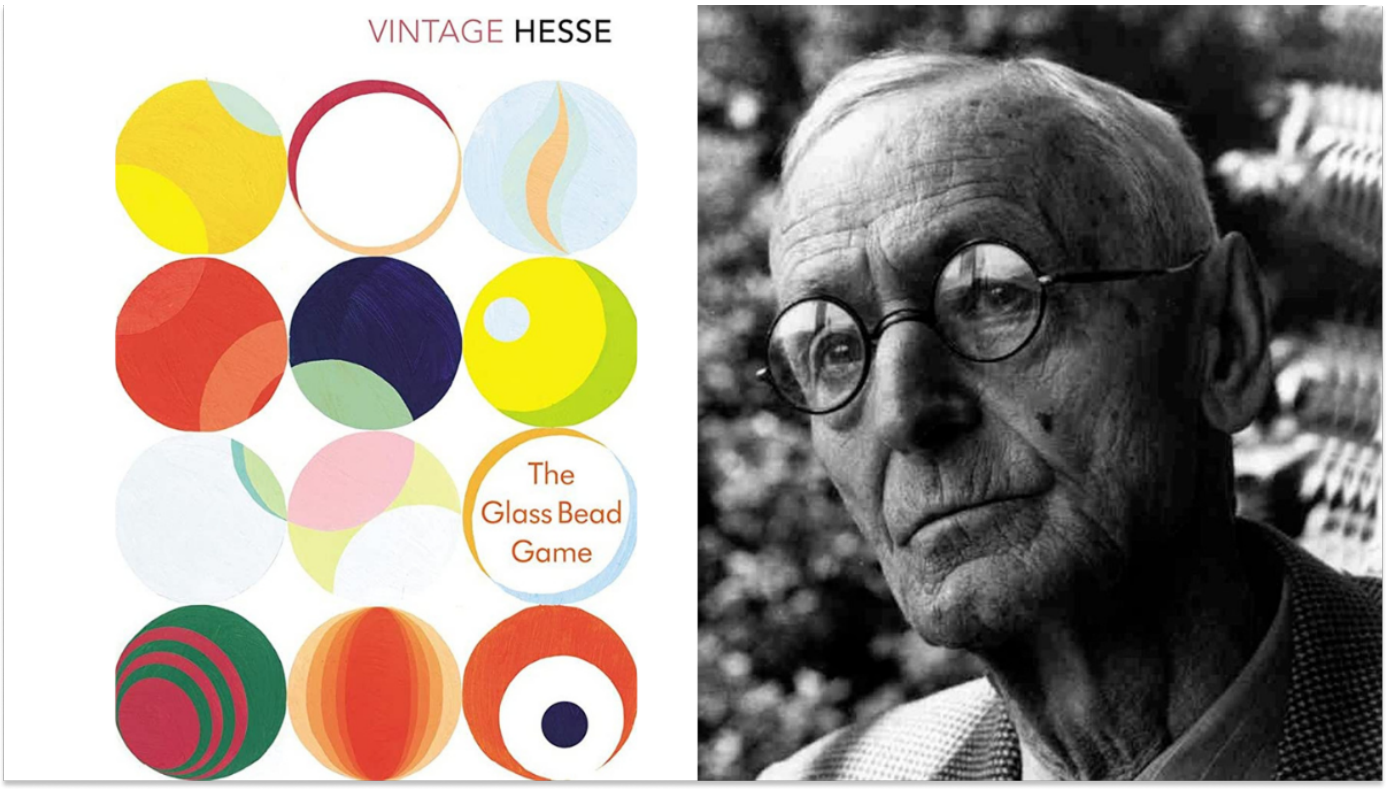
Some argue that code is the key to solving many real-world intelligence challenges because it can perfectly instruct all physical form-factors with precision.

We do not share that belief. A code-based simulation is a poor version of a dream. It is rule-bound and unable to handle the stochastic messiness of reality.

To know the world, you must interact with it.

In *The Glass Bead Game (Das Glasperlenspiel)*, a novel by Herman Hesse that won him the Nobel Prize for Literature in 1946, readers are introduced to Castalia, a future intellectual utopia devoted to pure thought. At Castalia's center is an elaborate game, the titular Glass Bead Game, that synthesizes all human knowledge into a single formal language. Players compose "games" the way one might compose a fugue. A

move might link a Bach cantata to a mathematical proof to a passage from Confucius. The game is the ultimate abstraction: all of human culture compressed into symbolic manipulation.



The protagonist, Joseph Knecht, rises to become Magister Ludi, Master of the Game, the highest position in Castalia. But he grows disillusioned. The game, for all its beauty, is *sterile*. Castalia's intellectuals have retreated so far into abstraction that they've lost touch with the world. They can *represent* reality with extraordinary elegance, but they cannot *act* in it.

Knecht ultimately decides he must leave Castalia, and becomes a simple tutor. He chooses the messy, embodied, unpredictable world over the perfect symbolic one. He dedicated his life to the Game, the mastery of which involves operating on a level of abstraction beyond words, something closer to world modeling. But it wasn't enough. Symbols alone, without contact with reality, eventually run dry.

Large Language Models are our Castalians. They are exquisite manipulators of symbols, capable of drawing connections across the entirety of human textual knowledge. They can discuss physics, compose poetry, write code, and explain the rules of baseball. They are, genuinely, one of the great intellectual achievements in human history.

But they operate entirely in the realm of representation. They can *describe* clapping, but they cannot clap. They can *talk* about gravity, but they do not know gravity the way a toddler knows gravity. They do not learn, the way a body learns, through thousands of falls and stumbles, what "down" means.

Language models predict the next token extraordinarily well. The only problem is that tokens are like shadows on Plato's cave wall. And you cannot code your way to a realistic stadium crowd any more than you can describe your way there.

The real world is — or *was* — **uncomputable**.

If language and code, two of mankind's most powerful inventions, are inadequate to represent our world, what do we have left?

The Answer is World Models

World Models offer another approach on the path to AGI. They offer a path to **compute the things that are, today, uncomputable**. They learn from the messy contact with reality that Knecht sought.

World Models offer a way to do non-deterministic compute efficiently, and to run simulations that shouldn't be possible under traditional compute constraints.

World models are not a replacement for LLMs. Language remains essential; text can be used to *condition* World Models, to tell them what scenario to imagine, what goal to pursue, to give them a long-term goal. The thinking and the doing work together. But the doing has to come from somewhere other than text.

Joseph Knecht must come down from Castalia.

Real intelligence must come from observation of the world; from understanding actions and their consequences; from the things that language can only point at.

The Dao that can be told is not the eternal Dao.

In the beginning was the Word. Then came humans, to act imperfectly and unpredictably.

Maybe this is the way of things. In the beginning were LLMs. Then came World Models.

What Are World Models?

A World Model simulates environments and responds when you act inside them.

More formally, a World Model is an interactive predictive model that simulates spatial-temporal environments in response to actions.

While LLMs predict the next word in a sentence, World Models predict the next state (as in, the immediate future), conditioned on the current state and control input.

More succinctly: **LLMs learn the structure of language. World Models learn the structure of causality.**

This is a simple definition of World Models. It is accurate, but it's not enough to understand how World Models work. For that, you'll need to know four things:

1. What World Models do,
2. How they're built,
3. Why "action" is so important, and
4. The relationship between World Models and policies.

What World Models Do

Think about what happens when you catch a ball. Your eyes take in a scene: the thrower's arm, the ball in flight, the wind, the sun in your eyes, all of it. From that flood of sensory data, your brain builds a compressed model of what's happening and, crucially, what's *about* to happen. It predicts the ball's trajectory a few hundred milliseconds into the future. Then it sends a motor command to your hand. You catch the ball. The whole loop — **observe, predict, act** — takes a fraction of a second and involves no language or "thinking" whatsoever.

A World Model does the same thing, computationally. It takes in observations (often video frames, though it can use any sensory data), builds a compressed internal representation of the environment's state, and predicts how that state will change in response to actions.

It is, in essence, a learned physics engine, but one that doesn't rely on hand-written equations. Instead of calculating gravity, collision, and friction from first principles, it has *watched* gravity, collision, and friction billions of times and learned the patterns.

This makes World Models a powerful tool for building **Agents**, AI systems that act in environments. World Models help Agents in three ways:

1. **They serve as surrogate training grounds.** An Agent can practice inside the World Model (basically, inside a dream) and transfer what it learns back to reality. This is important for safety (some things should not be tested or trained in the real world) and cost or sample/data efficiency (real world data is expensive, costly to gather, not available, you need a lot of it, etc.).
2. **They enable planning over longer time horizons.** An Agent can "imagine" the consequences of different actions before committing to one, the way a chess player thinks several moves ahead, except here, the board can be any environment or the real world.
3. **They provide rich representations of the world for Agents to learn behaviors from.** An Agent trained on a World Model's internal representations learns to "see" the world in terms of the features that matter for acting in it, rather than raw pixels.

For these three reasons, **the promise of World Models is that they are a path towards generalization.** If you can create worlds that respond to actions the way the real world does, you can use them to safely, economically, and efficiently train embodied agents that can act in any virtual world, or the real one.

To be clear, this is the massive question in World Models: whether the simulated environments are faithful enough to reality that you can train on them and have that training transfer to the real world or more generally, **whether you can "pre-train in sim."** Increasingly, the answer seems to be yes.

Ai2, the Allen Institute for AI, is a non-profit founded and funded by the late Microsoft co-founder, Paul Allen. It does great open source research and tooling, including its recent release of MolmoBot, an "open model suite for robotics, trained entirely in simulation."

"Our results show that sim-to-real zero shot transfer for manipulation is possible," they tweeted.

Dhruv Shah, a Princeton professor and Google DeepMind researcher who worked on the project, shared: "Within the scope of easily simulate-able tasks, a purely sim-trained policy outperforms SOTA VLAs trained on thousands of hours of real data!"

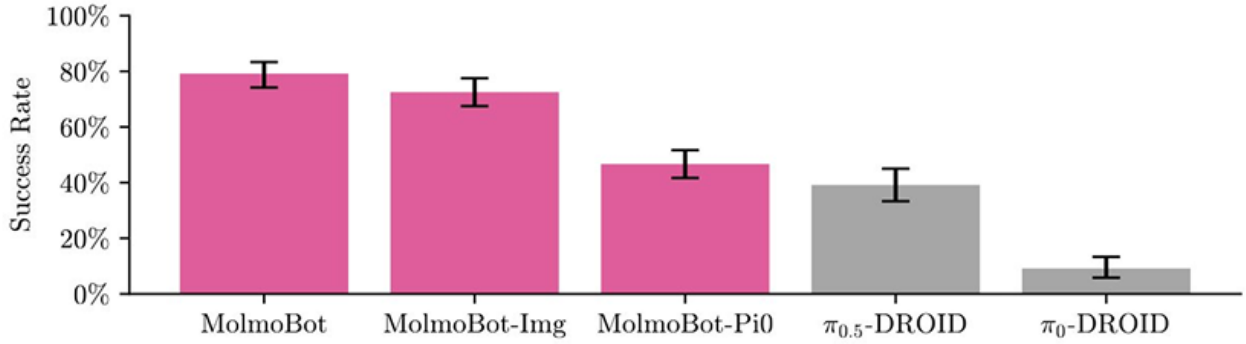


Figure 7 MolmoBot policies exhibit strong zero-shot sim2real performance across our real-world DROID evaluations, outperforming SOTA policies trained on large-scale real-world demonstrations. Bar heights reflect mean success rate and error bars represent 95% confidence intervals, estimated via stratified bootstrapping. Here, MolmoBot denotes the MolmoBot (F=2) variant.

Ai2, MolmoBot

It is a pretty astonishing finding. **A big focus of ours, and of the broader World Models field, is to expand the scope of tasks that are easy to simulate.**

This is how it works. First, World Models imagine realistic environments and future states, ideally that respond to actions or instructions in the way the real and virtual worlds they've been trained on do. Next, the Agents are let loose inside of the generated worlds to train. Then, the Agents are brought back into real environments and are tested on what they've learned.

This is what Ha and Schmidhuber demonstrated in 2018. It remains the central promise of the field.

How World Models Are Built

World Models are fairly young. No single approach or combination thereof has proved superior, which means that the final architecture for general World Models is still an open question. There are, however, repeatable ingredients for training.

Start with data; massive quantities of observation data. Often, observations are paired with the actions taken to produce them. This pairing can come about in several ways. Observations (typically video) are collected in advance and actions are either recorded alongside them, or inferred via another model after the fact. Alternatively, the model learns by taking actions itself, generating its own observations and action data through direct interaction with an environment.

When the training data is observations or videos, the raw frames serve as observations of an environment unfolding over time. These videos are ideally labeled with the actions that produced them (either because they were recorded together or inferred with a separate AI model). The actions provide the causal link: what someone did that made the environment change. A gameplay clip where a player turns left and the camera pans to reveal a hallway. A driving recording where the wheel turns and the car follows a curve. A teleoperation session where a robotic arm reaches and a cup moves. In each case, the model sees a before, an action, and an after.

When the model learns through interaction, the same structure applies — before, action, after — but the data is generated on the fly rather than collected in advance, and the actions come from the model's own developing policy rather than from an external source.

The World Model's core objective remains the same: **given the current state and an action or instruction, predict the next state.** It sees frame *t* and action *a*, and tries to produce state frame *t+1.*

But predicting raw pixel worlds for everything can be expensive and often wasteful. Most of what's in a video frame doesn't change from one moment to the next; the walls stay where they are, the sky remains the sky. And most of the details within a frame are redundant; the color of the sky, the texture of a wall. They could be described in a more compact form.

So modern World Models involve a ***latent* space**: a compressed, learned representation where *only the most essential information* is retained.

The visual encoder compresses each frame down to a compact vector (a mathematical fingerprint of the scene) and the model learns to predict the next fingerprint — not every pixel in the 4K frame — in response to actions. **This is where the computational efficiency comes from.**

To accurately model the evolution of the world, World Models must also learn to represent the full set of possible outcomes. This uncertainty in outcomes is usually referred to as the ***stochasticity*** of the environment.

World Models have to learn to navigate what they don't know yet (epistemic uncertainty: for example, a model that has never seen a traffic light will not know that red follows after yellow) and the inherently unknowable (aleatoric uncertainty: the randomness, like rolling dice[5]).

Even when the model has learned all that's possible to know about the behavior of the environment (it has reduced its "epistemic" uncertainty to a minimum), there will almost always be some inherent uncertainty ("aleatoric" uncertainty) in what happens next. This is in contrast to pure entertainment video models, which only need to be able to predict a common evolution of the world state to perform well.

If you use a straightforward prediction approach (for example, a model naively trained with Mean Squared Error, or MSE) to predict a car turning a corner, the model can become 'blurry' because it averages every possible outcome. The car could turn and stay in the left lane, or it could merge into the right lane. The trajectory that actually minimizes the error is the implausible one where the car stays in the middle of the two lanes. That's the blurriness, and different models handle it differently.

Diffusion models avoid this problem by gradually diffusing towards the outcome, enabling the model to commit to a specific mode of the outcome distribution, sampling a sharp, plausible future rather than averaging all possibilities.

Autoregressive models with multiple tokens per outcome also handle multimodality; by sampling one token after the other, they ensure that future token predictions are consistent with previous ones.

JEPA-style architectures, by contrast, address blurriness by simply sidestepping it. JEPA largely avoids having to model that distribution explicitly by never decoding back to pixel space at all. It operates in a space where averaging is less catastrophic, because we don't expect these models to predict frames, but rather to develop representations that are useful for downstream tasks.

What comes out of this process depends on what you need. If you're building a visual world simulator — something you can watch or explore — you decode the latent predictions back into pixels through a visual decoder, producing imagined video of plausible futures. This is what makes the demos from Google DeepMind and World Labs look realistic and impressive.

There are a number of approaches used to train World Models. We will cover them and how they evolved and built on each other through the lens of the brief eight-year modern history of the field shortly.

For now, keep this in mind: **observation data in, paired with the actions that caused what's happening in those observations, train World Models to predict the next state, Agents train to predict the next action in those Worlds.**

Why Actions are the Ultimate Form of Compression

Here is a key insight behind World Models: **actions are the ultimate form of compression.**

Consider what happens when you decide to step left to avoid a puddle. Your brain processes the visual scene (the sidewalk, the puddle, the people around you, the curb, the approaching bus), predicts the immediate future (the puddle won't move, the bus will pass, the person behind you will keep walking), evaluates options (step left, step right, jump, accept wet shoes), and selects one.

An outside observer can't see inside your head, can't know exactly what you were thinking, can't know what you're processing subconsciously. They don't know if you're tired or if you're in a rush. They don't know your moral code, how you, specifically, would answer the Trolley Problem. **They don't need to.** They see the output of all of that near-instantaneous calculation: step left.

That, to me, is magic.

Of course, not everyone makes the right decisions. Play the video forward and you are able to learn the consequences, too. Step left, into an even bigger puddle. Step left, and get clipped by a car. Step left, and knock a baby out of its stroller. Over billions and billions of observations and instructions and actions, we learn not just how humans decide to respond based on inputs, but the consequences of those decisions. **The collective World Model learns to act smarter than any individual.**

Zoom back into the individual. If you could perfectly reconstruct someone's stream of observations and actions, you would have a nearly complete record of their interaction with reality. You would know what they saw and what they did about it. **The World Model learns exactly this mapping.** It compresses space and time into a compact representation, and then uses actions to unroll what happens next. That's what makes World Models so computationally efficient.

It's also the same reason why World Models can handle stochasticity that traditional simulation cannot. To understand why, let's revisit our Man U match with our new understanding of how World Models work.

In a traditional simulation engine, every possible behavior must be coded. If you want a thousand soccer fans to react realistically to a goal, you need to write rules for each type of reaction. The computational cost scales with the number of Agents and the complexity of their interactions.

In a World Model, the cost is fixed to one neural network pass. The stochastic, messy, human reality is already baked into the learned weights and absorbed from the millions of hours of video the model was trained on. The model doesn't *calculate* what a crowd should do. It has seen what crowds *actually* do and it uses this information to make probable predictions.

This is what I mean when I call World Models compute for the uncomputable. Traditional computing is deterministic: known inputs, known rules, known outputs. The real world is not deterministic, so World Models don't even try to code these things in. They watch, learn, and do, at a fixed computational cost, regardless of how complex the scenario gets.

World Models and Policies

There is one more distinction to make before we go further, one that gets muddled in typical conversations about World Models.

A **World Model** is a simulation of the environment; it takes in actions and produces predicted observations; it shows you what will happen *if* you do something.

A **Policy** is the brains of the Agent that acts within that environment. It takes in observations (and often instructions) and produces actions; it decides what to do.

The World Model is the dream. The Policy is the dreamer. The dreamer acts, and the dream responds. The dream responds, and the dreamer acts.

In practice, the relationship between the two turns out to be even more intimate and intertwined than that distinction suggests. Recent research has investigated training policies on top of World Model foundations or building them together from the get go. Start with the weights of a World Model — a system that has learned how to predict what happens next — and then, instead of training the model to predict future frames, or states, you train it to predict future *actions*.

A system that learns to predict the world can also learn much faster how to act in it. Understanding and doing aren't two separate skills bolted together. They are the same skill, seen from different angles. At least this is what our research, and that of other labs, is starting to suggest.

That means that if you build a good enough World Model, you can also more effectively train a policy to act in the worlds it generates.

This is one of many important things the field has learned in a very short amount of time. Turns out intuition and imagination are two sides of the same coin.

A (Very Brief) History of World Models

On one hand, it should be very easy to summarize the modern history of World Models. It has only been eight years since Ha and Schmidhuber published *World Models.*

On the other hand, an awful lot has happened in just eight years. In that time, the field has gone through **four waves:** major periods when the field shifted its focus to prioritizing new questions. We highlight some of the most important papers here, and not boring world subscribers can find a full downloadable list of key papers at the end of the essay.

World Model Waves

Wave	Years	Question	Key Papers
Wave 0	1990-91	What would a World Model do?	*Dyna* *Making the World Differentiable*
Wave 1	2018-19	Can this even work?	*World Models* *Model-Based Reinforcement Learning for Atari / SimPLe*
Wave 2	2020-22	Can the World Model match human performance?	*DreamerV2* *MuZero* *IRIS* *A Path Towards Autonomous Machine Intelligence / JEPA*
Wave 3	2023-24	Can World Models be truly interactive?	*GAIA-1* *DIAMOND* *Genie*
Wave 4	Present	Can World Models act in the real world?	*GAIA-2 and GAIA-3* *V-JEPA 2* *SIMA 2* *Learning to Drive from a World Model*

not boring general intuition

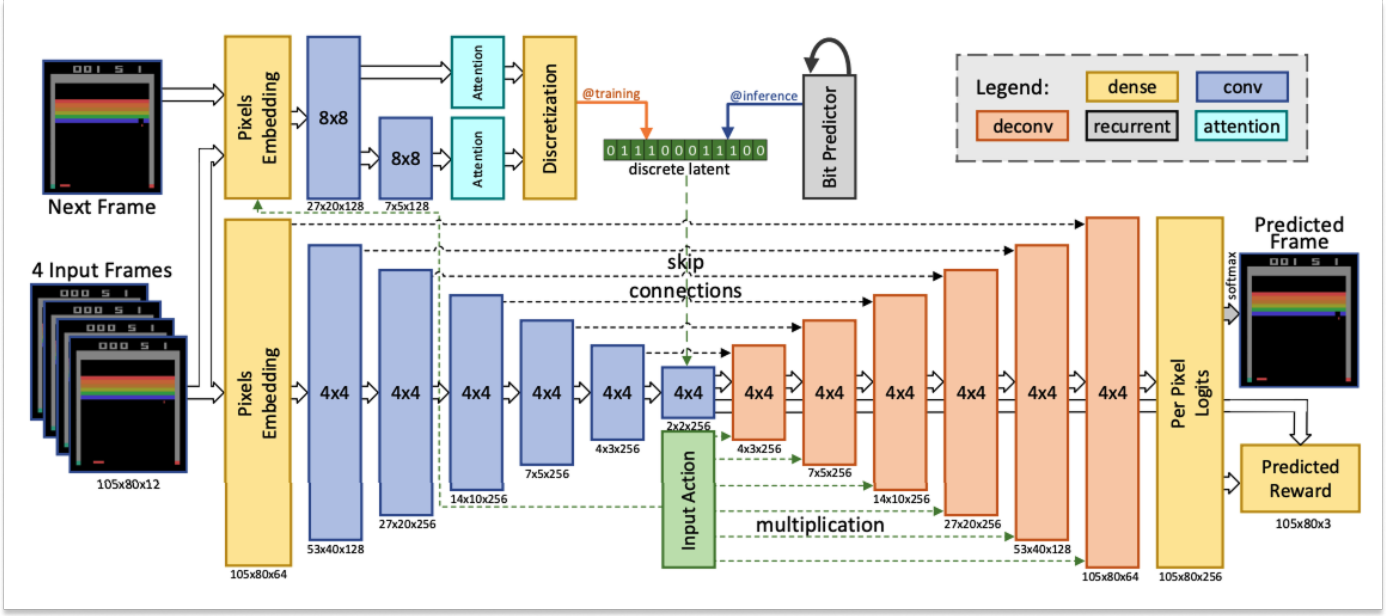
Wave 0, in 1990-1991, was the pre-deep learning era. Researchers first articulated the idea that Agents could learn internal models of the world and use them for prediction and planning. They asked, and answered, the question: what would a World Model do?

This is Richard Sutton and *Dyna*. This is Jürgen Schmidhuber and *Making the World Differentiable*. Before we had the compute, the data, or the architecture, we had the dream, waiting in dreamspace for reality to catch up.

Wave 1, in 2018-2019, asked: **"Can this even work?"**

Based on Ha and Schmidhuber's work, the first paradigm involved using **Video Auto-Encoders (VAE)** to compress frames, model dynamics with **Recurrent Neural Networks (RNN)**, and train policies inside the resulting dreams. So: compress what you see, predict what comes next, and train Agents to act inside that simulation.

At the time, the question was whether learning in imagination — dreams — was feasible. Researchers attempted to answer it using small models and simple environments to generate proof-of-concept results. Quite literally, the next big thing started out looking like a toy. *Model Based Reinforcement Learning for Atari* introduced the Atari 100k benchmark: whether the SimPLe algorithm could learn Atari games with only 100,000 real environment steps, or about two hours of gameplay.



The World Model Inside of SimPLe

The answer was yes. SimPLe learned how to play 26 Atari games and beat a competitor model on **sample efficiency**, or how many steps it took to reach a given score.

But could it play as well as humans?

That was the question that drove **Wave 2** (2020-2022): **"Can the World Model match human performance?"**

DreamerV2, developed by Danijar Hafner at Google DeepMind, reached an answer quickly. They used a **Recurrent State-Space Model (RSSM)** with discrete latent representations — a system that maintains a compressed, running memory of the world and updates it with each observation. DreamerV2 became the first World Model Agent to achieve human-level performance across the 55-game Atari benchmark6. It was trained entirely in imagination, on a single GPU.

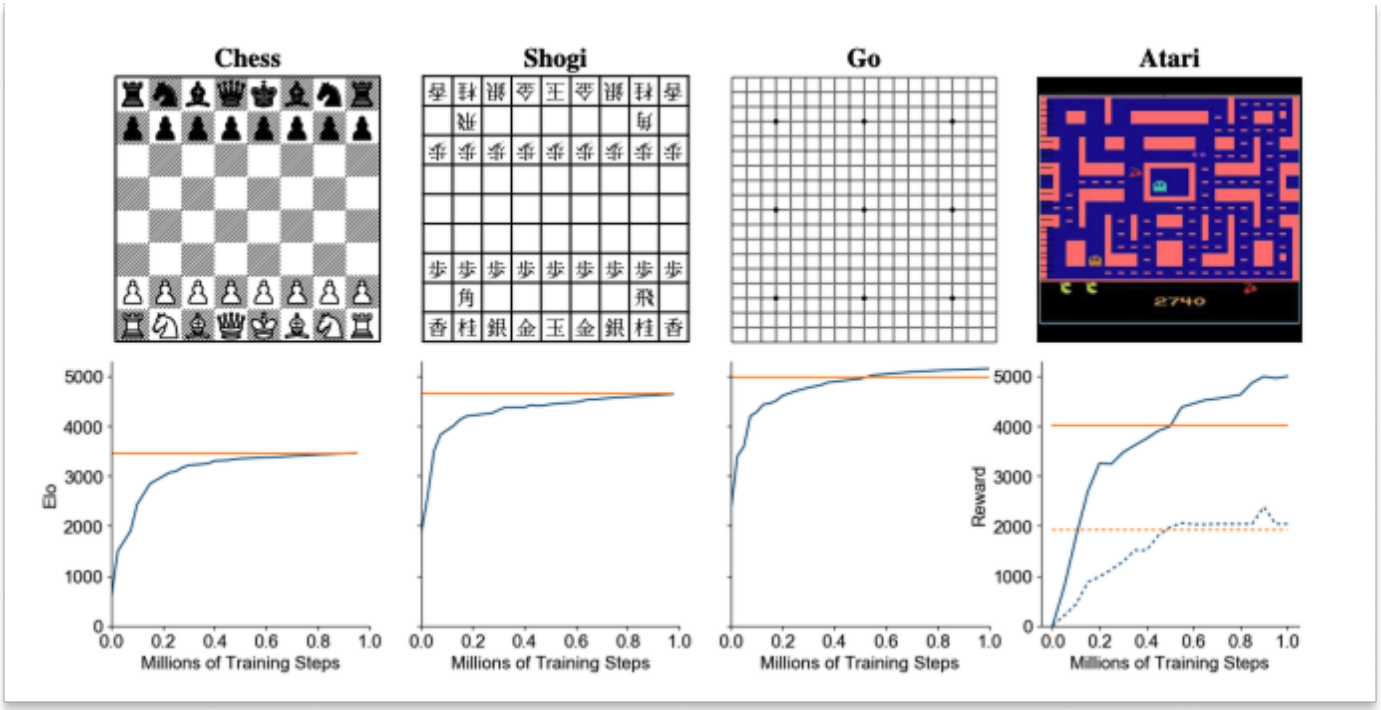
That same year, another DeepMind team published *Mastering Atari, Go, chess and shogi by planning with a learned model* in *Nature*. The paper described its **MuZero** model, which also beat Atari games (and others like Go), but did so by taking almost the exact opposite philosophical approach.

Algorithm	Reward Modeling	Image Modeling	Latent Transitions	Single GPU	Trainable Parameters	Atari Frames	Accelerator Days
DreamerV2	✓	✓	✓	✓	22M	200M	10
SimPLe	✓	✓	✗	✓	74M	4M	40
MuZero	✓	✗	✓	✗	40M	20B	80
MuZero Reanalyze	✓	✗	✓	✗	40M	200M	80

Comparison From the DreamerV2 Paper

Whereas DreamerV2 generated observable dream environments and trained inside of them, *MuZero never generated anything observable at all*, planning entirely in abstract latent representations it invented for itself, and it did well.

It did so well, in fact that it leapfrogged the Go-specific models. In 2016, DeepMind's **AlphaGo** beat human Go Champion Lee Sedol 4-1. It had been trained on a large database of human expert games plus self-play, with the rules of the game hard-coded in. The next year, **AlphaGoZero** beat AlphaGo 100-0 after being trained entirely from self-play with no human game data at all, just the rules. That same paper season, **AlphaZero** generalized AlphaGoZero's approach to other games, like chess and shogi, both of which it came to dominate within hours. Then in 2019 (pre-print), **MuZero** learned everything, including the rules, the game dynamics, and the value function, from scratch, purely from observation and outcome. It matched AlphaZero on Go, chess, and shogi (where AlphaZero knew the rules) while also generalizing to 57 Atari games (where "rules" aren't even a well-defined concept).



MuZero

With each new model, something that humans had previously hard-coded — the rules, the strategy, the value of a position — was removed. The model learned each from scratch instead. MuZero was the terminus of that progression, entirely learned.

And MuZero did this without imagining future board states at all. It imagined hidden states, or abstract vectors it invented for itself during training that have no guaranteed correspondence to anything human-observable or interpretable. A human looking at MuZero's internal representation of "three moves from now" would have absolutely no idea what it was thinking. And yet... it outperformed all previous models.

With MuZero's success, the field now had two opposing schools of thought: **generative World Models that produce observable futures, and latent World Models that predict in abstract space**, even if they weren't called "latent" yet.

From then on, progress in World Models has happened in both directions, generative and latent.

On the latent side, in 2022, Yann LeCun published a sweeping position paper from his dual positions at Meta and NYU Courant proposing a fundamentally different philosophy from generative models, one that looked more like MuZero: *A Path Towards Autonomous Machine Intelligence*. His new World Models company, AMI, is named after this paper.

A Path Towards Autonomous Machine Intelligence
Version 0.9.2, 2022-06-27

Yann LeCun
Courant Institute of Mathematical Sciences, New York University yann@cs.nyu.edu
Meta - Fundamental AI Research yann@fb.com

June 27, 2022

Abstract

How could machines learn as efficiently as humans and animals? How could machines learn to reason and plan? How could machines learn representations of percepts and action plans at multiple levels of abstraction, enabling them to reason, predict, and plan at multiple time horizons? This position paper proposes an architecture and training paradigms with which to construct autonomous intelligent agents. It combines concepts such as configurable predictive world model, behavior driven through intrinsic motivation, and hierarchical joint embedding architectures trained with self-supervised learning.

LeCun's **Joint Embedding Predictive Architecture (JEPA)** argued *against generating pixels entirely*. Similar to MuZero, instead of predicting what the world will *look like*, JEPA predicts what it will *mean*. It forecasts abstract representations of future states, deliberately discarding unpredictable visual details.

That same year, on the generative side, IRIS (2022), developed by Vincent Micheli and Eloi Alonso, two of General Intuition's future co-founders, reframed World Modeling as language modeling over a learned vocabulary of image tokens. Instead of recurrent state-space models, IRIS used a GPT-style autoregressive transformer over discrete visual tokens. Basically, IRIS borrowed the machinery of language models and applied it to World Modeling.

In doing so, IRIS filled a number of previous gaps. The IRIS World Model was, in effect, a language model, but its vocabulary was images and actions instead of words. This brought the **scaling** properties of LLMs directly into World Modeling: efficient attention, scaling laws, and all the engineering infrastructure that had been built for large language models could now be applied to learning about the physical world.

Where Dreamer was missing the ability to model the joint law of the next latent state (for example, to handle multimodality), IRIS represented the next latent state as a series of discrete tokens to predict autoregressively, which meant that it was now able to predict multiple outcomes. And while Dreamer beat humans by using much more data than they do, IRIS was the first learning-in-imagination approach to beat humans with the same amount of available gameplay data (two hours).

Table 1: Returns on the 26 games of Atari 100k after 2 hours of real-time experience, and human-normalized aggregate metrics. Bold numbers indicate the top methods without lookahead search while underlined numbers specify the overall best methods. IRIS outperforms learning-only methods in terms of number of superhuman games, mean, interquartile mean (IQM), and optimality gap.

			Lookahead search		No lookahead search				
Game	Random	Human	MuZero	EfficientZero	SimPLe	CURL	DrQ	SPR	IRIS (ours)
Alien	227.8	7127.7	530.0	808.5	616.9	711.0	**865.2**	841.9	420.0
Amidar	5.8	1719.5	38.8	148.6	74.3	113.7	137.8	**179.7**	143.0
Assault	222.4	742.0	500.1	1263.1	527.2	500.9	579.6	565.6	**1524.4**
Asterix	210.0	8503.3	1734.0	25557.8	**1128.3**	567.2	763.6	962.5	853.6
BankHeist	14.2	753.1	192.5	351.0	34.2	65.3	232.9	**345.4**	53.1
BattleZone	2360.0	37187.5	7687.5	13871.2	4031.2	8997.8	10165.3	**14834.1**	13074.0
Boxing	0.1	12.1	15.1	52.7	7.8	0.9	9.0	35.7	**70.1**
Breakout	1.7	30.5	48.0	414.1	16.4	2.6	19.8	19.6	**83.7**
ChopperCommand	811.0	7387.8	1350.0	1117.3	979.4	783.5	844.6	946.3	**1565.0**
CrazyClimber	10780.5	35829.4	56937.0	83940.2	**62583.6**	9154.4	21539.0	36700.5	59324.2
DemonAttack	152.1	1971.0	3527.0	13003.9	208.1	646.5	1321.5	517.6	**2034.4**
Freeway	0.0	29.6	21.8	21.8	16.7	28.3	20.3	19.3	**31.1**
Frostbite	65.2	4334.7	255.0	296.3	236.9	**1226.5**	1014.2	1170.7	259.1
Gopher	257.6	2412.5	1256.0	3260.3	596.8	400.9	621.6	660.6	**2236.1**
Hero	1027.0	30826.4	3095.0	9315.9	2656.6	4987.7	4167.9	5858.6	**7037.4**
Jamesbond	29.0	302.8	87.5	517.0	100.5	331.0	349.1	366.5	**462.7**
Kangaroo	52.0	3035.0	62.5	724.1	51.2	740.2	1088.4	**3617.4**	838.2
Krull	1598.0	2665.5	4890.8	5663.3	2204.8	3049.2	4402.1	3681.6	**6616.4**
KungFuMaster	258.5	22736.3	18813.0	30944.8	14862.5	8155.6	11467.4	14783.2	**21759.8**
MsPacman	307.3	6951.6	1265.6	1281.2	**1480.0**	1064.0	1218.1	1318.4	999.1
Pong	-20.7	14.6	-6.7	20.1	12.8	-18.5	-9.1	-5.4	**14.6**
PrivateEye	24.9	69571.3	56.3	96.7	35.0	81.9	3.5	86.0	**100.0**
Qbert	163.9	13455.0	3952.0	13781.9	1288.8	727.0	**1810.7**	866.3	745.7
RoadRunner	11.5	7845.0	2500.0	17751.3	5640.6	5006.1	11211.4	**12213.1**	9614.6
Seaquest	68.4	42054.7	208.0	1100.2	**683.3**	315.2	352.3	558.1	661.3
UpNDown	533.4	11693.2	2896.9	17264.2	3350.3	2646.4	4324.5	**10859.2**	3546.2
#Superhuman (↑)	0	N/A	5	14	1	2	3	6	**10**
Mean (↑)	0.000	1.000	0.562	1.943	0.332	0.261	0.465	0.616	**1.046**
Median (↑)	0.000	1.000	0.227	1.090	0.134	0.092	0.313	**0.396**	0.289
IQM (↑)	0.000	1.000	N/A	N/A	0.130	0.113	0.280	0.337	**0.501**
Optimality Gap (↓)	1.000	0.000	N/A	N/A	0.729	0.768	0.631	0.577	**0.512**

IRIS Results

JEPA aside, practically all of the work up to this point in World Models happened within **games**, and it's worth floating between Wave 2 and Wave 3 for a second to appreciate the special relationship between AI and games.

Games have always played an important role in the development of AI. Claude Shannon's 1950 paper *Programming a Computer for Playing Chess* is one of the founding documents of AI. In 1959, Arthur Samuel's checkers program introduced the concept of machine learning itself. The first time the world woke up to the idea that intelligent machines could beat humans at anything was when IBM's Deep Blue beat Garry Kasparov in chess.

Garry Kasparov (l), dejected

Before DeepMind was an AI lab, Demis Hassabis was a game designer. At 17, he designed the commercially successful *Theme Park*. DeepMind's founding breakthrough is detailed in the DQN paper, published in *Nature* in 2015, in which it was demonstrated that Atari games could be played from raw pixels using deep reinforcement learning. Then came AlphaGo in 2016, which beat the world champion at Go, a game that once was believed to require the kind of intuition that was uniquely human, with more possible board positions than atoms in the universe.

The path from AlphaGo to AlphaFold ran through exactly the insight that World Models formalize. As Hassabis put it:

> ***Wouldn't it be incredible if we could mimic the intuition of these gamers, who are, by the way, only amateur biologists?***

General Intuition is named after this quote from Demis, which points towards a future where our models power research far beyond the dynamics of what pixels can describe today, beyond games themselves, and into our bodies.

And then DeepMind taught machines how to fold proteins. AlphaFold won Hassabis and his DeepMind teammate John Jumper the 2024 Nobel Prize in Chemistry.

Games are fun, of course. **But the reason games keep showing up is that games are the only domain where you get massive amounts of labeled spatial-temporal data with clear action-outcome pairs, consistent physics, unambiguous reward signals, and a controlled environment where you can run millions of experiments.** The real world has none of these properties.

Early World Models, like a human child, spent most of their time watching and playing games. The Atari 100k benchmark became the standard arena for World Model research, DreamerV3 played Minecraft, and many current World Model companies retain a connection to games, with many World Models being "playable."

Games are the lab bench of embodied AI. But they are only a small fraction of the ambition.

For World Models to be truly useful, they need to interact with the world.

That's **Wave 3** (2023-2024). It asked: **"Can World Models be truly interactive?"**

We got the first answer from driving. GAIA-1 (2023), developed at Wayve, scaled the sequence-modeling approach pioneered by IRIS to 9 billion parameters and trained on real-world driving video. It could generate driving scenarios in response to actions (steer the car), text prompts ("rainy day, highway"), or both. Anthony Hu, who led this research, now leads World Modeling at General Intuition.

GAIA-1 confirmed that the scaling laws everyone had observed in LLMs also held for visual World Models. More data and more parameters yield predictably better performance for World Models, too. This was not a given. It meant that the path forward was clear even if it was expensive: scale up and the models get better.

The following year, DIAMOND (2024), developed by future General Intuition co-founders Eloi Alonso, Adam Jelley, and Vincent Micheli, opened a new architectural frontier. Rather than compressing observations into discrete tokens and predicting them autoregressively, as researchers had been doing since IRIS, DIAMOND **used diffusion models to predict future frames directly**.

The visual fidelity was meaningfully richer, and that richness translated directly into better Agent performance. The subtle visual details that discrete tokens discarded, the little clues that tell you a surface is slippery, a door is ajar, a person is about to change direction, turned out to matter for decision-making, which is unsurprising when you think about it.

As a brief aside, it's worth noting that many of the open source advancements that have been made in World Modeling were built on top of the DIAMOND architecture. Multiverse, the first AI-generated multiplayer game, is DIAMOND-based, as is Alakazam, the "1st 'World Model game engine'." DIAMOND is essentially the Deepseek or Llama of Generative World Models.

DIAMOND itself set a new best on Atari 100k and demonstrated something that captured the public imagination: trained on Counter-Strike gameplay, it produced a fully interactive, playable neural game engine from roughly 87 hours of footage on a single GPU.

It showed that it was possible to run an interactive 3D World Model in real time too.

Counter-Strike environment generated by DIAMOND with just 87 hours of footage

DIAMOND got very good at playing Atari. The Agent plays a real game and gathers real data there, with which it trains the World Model. Then it tests itself inside the World Model's synthetic environment, gets better in there, and goes back out for more real interaction, to test itself in the wild. This loop between ground truth and synthetic, back and forth, is how World Models improve, almost like working problems out in a lucid dream then testing them in reality upon waking. This is the Dyna paradigm mentioned earlier.

Would that loop work in real-world conditions?

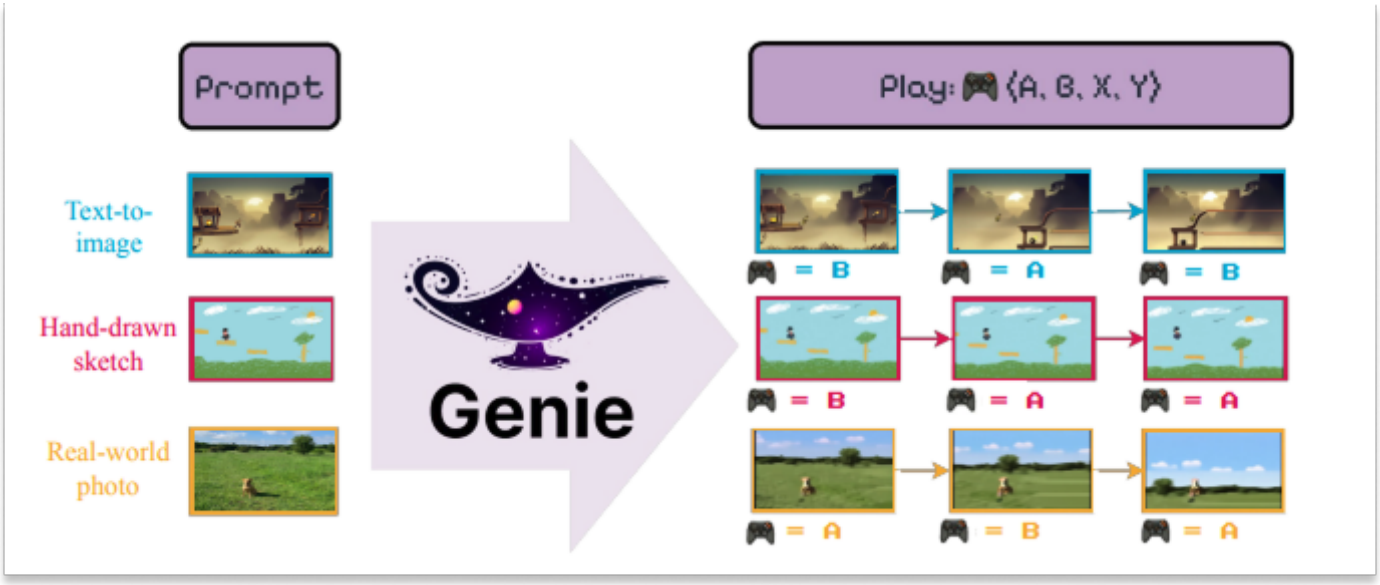
It turns out that the answer is yes, too. And that it would work beautifully.

GAIA-2 (March 2025) pushed the diffusion approach to its most ambitious application yet: multi-camera autonomous driving simulation. Using latent diffusion with flow matching and space-time factorized transformers, the model could generate high-resolution surround-view driving video conditioned on ego-vehicle dynamics, other Agents' trajectories, weather, time of day, road structure. In short, **it could reproduce the full complexity of real driving**. It could simulate scenarios that were too dangerous or too rare to collect from real roads: sudden cut-ins, emergency braking, pedestrians stepping off curbs.

GAIAs 1 and 2, and DIAMOND, like IRIS, were the products of researchers we now get to work with at General Intuition. Diffusion or flow-matching models like GAIA-2 were the starting point of our team's current research efforts.

But they are not the only approach.

Google DeepMind is one of the central players in this space. Their World Model, Genie (2024), is an 11-billion-parameter model trained on unlabeled internet video of 2D platformer games. It learned an action space entirely from scratch; no one ever told the model what the controls were. Give it any image and it can generate a playable world from it.



Genie: A Whole New World

OpenAI's Sora (2024, with Sora 2 following in 2025) and Google's Veo 3 (2025) pushed video generation to extraordinary visual quality and framed these systems explicitly as "world simulators."

The field's vocabulary can get muddied. Let's make it clear.

Video generation models produce beautiful visual sequences, but they aren't quite World Models in the sense that we've been describing them. In these videos, you can't take an action and watch the environment respond live to your intervention. They predict what a scene will look like over time; they don't model what happens because of what you *do*.

Think of the difference between watching a movie of someone driving and actually steering a car. The visual output might look similar, but the underlying computation is fundamentally different. **Interactivity**, the ability to take actions and observe their consequences, is what separates a World Model from a very impressive video.

And interactivity is what it takes to impact the real world.

This is the central question of **Wave 4**, the wave we're in *right now*: **"Can models act in the real world?"**

As in: Can Agents trained in World Models work outside of research settings, in real vehicles, real robots, real deployments? We are now getting awfully close to sci-fi's predictions.

This is where the current frontier is being pushed. Right now. As you read this.

Comma.ai took the most direct path in driving from World Model to product: *Learning to Drive from a World Model*. They trained a driving policy *entirely inside a learned World Model* — inside the dream — and deployed it in openpilot, their open-source driver assistance system running on production vehicles driven by real people. The World-Model-trained policy outperformed both traditional imitation learning and policies trained in conventional simulators. This is arguably the first consumer product powered by a World-Model-

trained Agent.

In robotics, Meta's V-JEPA 2 animated LeCun's latent prediction philosophy. The model is the clearest large-scale proof point so far. It's a 1.2B-parameter model pre-trained on over a million hours of video via self-supervised masked prediction: no labels, no text. In the second stage, it fine-tunes on just 62 hours of robot data from the Droid dataset. It turns out that is enough to produce an action-conditioned World Model that supports zero-shot planning. V-JEPA 2 was deployed zero-shot on real Franka robot arms in new environments to perform pick-and-place tasks. It *planned all of this entirely in latent space*, without pixel generation, task-specific training, or hand-crafted rewards. And it was *fast*; where pixel-space approaches took minutes to plan a single action, V-JEPA 2 did it in seconds.

Google DeepMind's SIMA 2 took an entirely different approach. Rather than build a dedicated World Model, it fine-tuned Gemini, its large foundation model, to act directly as an Agent in 3D game environments. SIMA 2 can reason about high-level goals, follow complex multi-step instructions, converse with users, and generalize to unseen environments.

It represents an alternative paradigm: instead of building a specialized World Model, leverage the implicit world knowledge already embedded in a model trained on the breadth of human knowledge.

This is one of the field's open questions. Will this path, using a large foundation model or a video model as the basis for an Agent, rather than training an Agent from scratch in a World Model, win out?

In fact, there are many open questions. And nearly as many World Model startups trying to answer them.

The State of the World (Models)

That brings us to the present moment.

What has become clear is that talented researchers and investors alike are excited by World Models' potential, as evidenced by the massive funding rounds to support companies led by legends in the field.

In February 2026, World Labs, the company founded by legendary researcher Fei-Fei Li, announced that it had raised a fresh $1 billion from investors at a $5.4 billion post-money valuation.

Not to be outdone, Yann LeCun, who launched AMI Labs in late 2025, announced last week that it had raised $1.*03* billion at a $3.5 billion valuation.

In October 2025, our company, General Intuition, announced $133.7 million in a very large seed round. Last summer, Decart raised $100 million at a $3.1 billion valuation. In November, Physical Intelligence raised $600 million at a $5.6 billion valuation for its robot foundation models. And just this past February, Wayve, the UK-based self-driving startup whose researchers built GAIA-1 and GAIA-2, raised $1.2 billion at an $8.6 billion valuation.

Google DeepMind, which doesn't need to fundraise because it's fueled by history's greatest business machine, is pouring resources into SIMA, Genie, and Veo, and using it to power initiatives like Waymo. Demis has publicly stated that he believes World Models will become an important part of Gemini's planning capabilities. GDM is also merging many of these capabilities into a "Video Thinking" team, with the reasoning described best by Shane Gu and Jack Parker Holder from GDM.

World Model and World Model-Adjacent Companies Have Raised Billions

Company	Founded	Key Founders / Leaders	Last Announced Funding	Valuation
General Intuition	2025	Pim de Witte, Adam Jelley, Eloi Alonso, Vincent Micheli	$134M Seed (Oct 2025)	—
AMI Labs	2026	Yann LeCun (Exec Chair), Alex LeBrun (CEO)	$1.03B Series A (Mar 2026)	~$3.5B
World Labs	2024	Fei-Fei Li, Justin Johnson, Ben Mildenhall	$1B Series B (Feb 2026)	~$5.4B
Google DeepMind	2010	Demis Hassabis (CEO)	Alphabet subsidiary	—
Moonlake AI	2024	Chris Manning, Ian Goodfellow, Fan-Yun Sun	$28M Seed (2025)	—
Decart	~2023	Dean Leitersdorf	$100M (2024)	~$3.1B
Odyssey	~2024	Oliver Cameron, Jeff Hawke	$18M Series A	—
Wayve	2017	Alex Kendall, Amar Shah	$1.2B (Feb 2026)	~$8.6B
Physical Intelligence	2024	Karol Hausman, Sergey Levine, Chelsea Finn, Lachy Groom	$600M (Nov 2025)	~$5.6B
Runway	2018	Cristóbal Valenzuela	$315M Series C (Feb 2026)	~$5.3B

Selected company data based on publicly available data as of 3/18/2026

not boring

What is less clear, but even more interesting, is that we are at the point in this technology's development where we know that *something* big is happening, but it's still unclear exactly *which* approach, or *combination* of approaches, will win out. We are seeing breakthroughs almost every day at General Intuition, and we hear rumors of leaps happening in other labs too.

Below is a framework in which to fit any news you see coming out on World Models. We won't cover everything, and we apologize in advance if we miss your embodied AI of choice. A fun exercise for the reader will be to fit what we've missed into what we've laid out.

World models have three main types of approaches: **Current Foundation Models, World Models, and Embodied Agents.**

The thing to keep in mind here is that, despite different World Model approaches, we all share the same end goal. **The end goal is to produce Agents that generalize and do things in various environments, including the real world.** Some of the Agent approaches get there using LLMs as their stepping stone, others start with video models. Other agent approaches use World Models as their training environments. And some Agents learn directly from experience.

With us? Goed, daar gaan we dan!

Current Foundation Models

Current Foundation Models are the ones that learned to make sense of the world's data without being able to simulate the stochastic world environment itself. They are models that process inputs — text, images, video — and learn to predict, generate, or reconstruct. But they don't yet give an Agent a place to act. They are not action-conditioned. They don't respond or interact. They are potential substrates on which World Models can be built, or even, in some cases, on which Agents are pre-trained.

Three categories of stepping stone models we'll focus on here are **Large Language Models, Video Models, and 3D Reconstruction Models.**

Large Language Models

LLMs learned from staggering quantities of text that the world has structure. They know that a glass falls when pushed, that fire is hot, that if you leave the house without an umbrella in a rainstorm you will get wet. They encoded an enormous amount of causal and physical knowledge. But none of this was from experience. Like digital Castelians, they read about the world rather than perceiving it. This makes them

extraordinarily useful as a backbone for reasoning and planning, which is why you'll find LLMs embedded in many agent architectures we'll discuss later. But a language model alone cannot simulate what happens when a robot arm reaches for a cup.

In our context, LLMs are particularly relevant when we discuss **VLAs**, or Video Language Action models, which take advantage of the enormous amount of research, capital, tooling, and infrastructure that has gone into developing LLMs in order to bootstrap robots that can do things in the physical world.

Video Models

Sora. Veo 3. Kling. Seedance 2.0. Runway. Pika. Moonvalley. Haiper. Luma AI.

No one confuses an LLM for a World Model, but plenty of people conflate Video Models and World Models.

These models are trained on the enormous amount of video data on the internet, and produce extraordinary videos themselves. Sora can generate a convincing shot of a woman walking through a neon-lit Tokyo street. Veo 3 can render photorealistic scenes with synchronized dialogue.

But you can't interact with them. You can't take an action inside of them and watch the environment respond instantly. They predict what a scene will *look like* over time but they don't try to model what happens *because of what you do*.

Of course, the lines get blurry.

Odyssey, founded by self-driving heavyweights Oliver Cameron (ex-Cruise) and Jeff Hawke (ex-Wayve), is building "a world simulator that dreams in video." Currently, they don't let you take an action and watch the environment respond, but they do let you prompt the video mid-stream to steer it in real-time. Where do you draw the line?

Wherever the line is, these video models are getting good, and really funny.

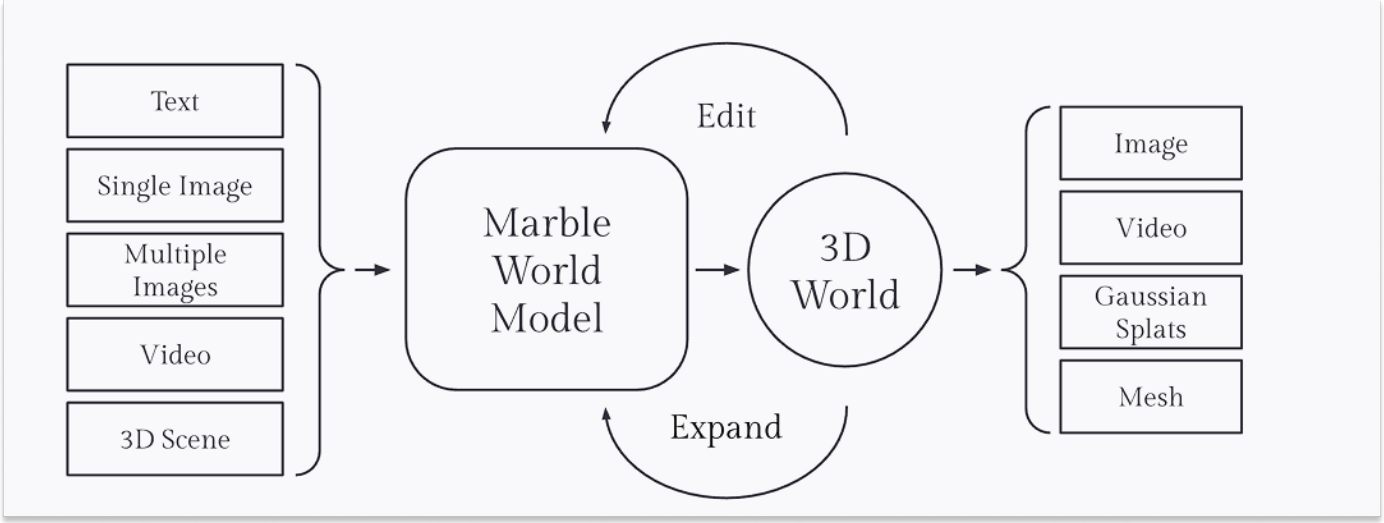
Video models aren't quite World Models in the sense we define them; they are a stepping stone. Runway began as a video generation company – its Gen 4.5 is among the best on the market – but has concluded that physics-aware video generation is a path toward something bigger. This thinking led to GWM-1, their explicitly labeled "General World Model, built to simulate reality in real time," which is interactive, controllable, and general-purpose. The real value, financially and societally, won't come from video for its own sake, but from models that use video as a training environment on the way to controlling embodied systems.

3D Reconstruction and Generation Models

Take it a step further. What if you could navigate through the scenes depicted in video generation models? That feels like a world, right?

World Labs, led by the legendary Fei-Fei Li, the "Godmother of AI" who created ImageNet, is the most interesting example in this category. While the company is the one most people would associate with "World Models," World Labs is not currently building what I would define as World Models.

Instead, in its early days, World Labs has focused on immersive virtual worlds, but not action-conditioned ones. Its first product Marble generates and edits persistent 3D environments from text, images, video, or 3D layouts. They call it a "Multimodal World Model."



World Labs

Marble is thus far not interactive, other than being able to move through the generated environments. They say this themselves. On the Marble product page, World Labs frames interactivity as a future opportunity:

> *Future World Models will let humans and agents alike interact with generated worlds in new ways, unlocking even more use cases in simulation, robotics, and beyond.*

It is worth noting that World Labs has recently started exploring World Models that do generate frames directly, instead of the underlying splats of the entire world.

World Models

A World Model, as we define it, is an environment that an Agent can act in, and that responds in real time. It is a simulation, a dream, one learned from observation and actions data rather than hand-coded. The Agent takes an action, the world changes, and the Agent observes what happened. Repeat, millions of times, across an enormous variety of situations, and the hope is that you get an Agent that generalizes, that can do things that were not in the original training data.

This is the key distinction that everything else hinges on: **a World Model is action-conditioned**. It predicts what the world will look like next given whatever the Agent did.

The intuition is simple. A robot trained only on real-world data has seen a finite set of kitchens, a finite set of cups, a finite set of ways a cup can fall. Put it in a kitchen it hasn't seen, with a cup it hasn't encountered, and it struggles. A robot trained inside a World Model, on the other hand, has, in principle, encountered infinite kitchens because the World Model can generate them. Situations that would be rare, expensive, or dangerous to collect in the real world become routine in simulation. Out-of-distribution becomes in-distribution.

Within World Models, there are two main approaches: **Latent World Models** and **Generative World Models**.

I apologize for bringing you so far in the weeds here, but I want to clarify something that confuses people: both Generative World Models and Latent World Models rely on latent states, but Generative World Models rely on latent states that were designed with reconstruction objectives (autoencoders) which enable frame predictions, whereas Latent World Models directly build self-predictive representations.

Latent World Models were born in the darkness and still live there; Generative World Models were merely born in the darkness.

Latent World Models

Latent World Models are the descendants of MuZero but let loose in open-ended, no-rules environments like the real world.

This is Yann LeCun's current world. Yann pioneered modern computer vision architectures with LeNet, where he introduced the idea behind **convolutional neural nets (CNNs)** in the 1990s. In the 2010s, he championed **self-supervised learning**, arguing that human labeling millions of examples doesn't scale to real intelligence and that models should create their own signal from raw data. In the 2020s, he led the **JEPA** team. Yann is a GOAT.

The deep thread in Yann's work is teaching models to learn useful representations of the world automatically from raw data. Latent World Models are the latest, and perhaps ultimate, strand in this thread.

The approach is philosophically the converse of Video Models or 3D Reconstruction Models, as mentioned earlier in the history section. While those approaches care about producing and understanding every pixel, latent World Models, like JEPA, says *ne vous embêtez pas*. The French would rather speak English to you than listen to you butcher their language. JEPA is similarly impatient; rather than let the model stumble over every pixel of an unpredictable future, it doesn't predict pixels at all.

As LeCun puts it: “The world is unpredictable. If you try to build a generative model that predicts every detail of the future, it will fail. JEPA is not generative AI.”

Instead, JEPA learns to represent videos in abstract, compressed space and makes predictions there. It deliberately throws away unpredictable visual details. This makes JEPA potentially very efficient for planning and representation learning.

AMI Labs is LeCun’s bet that this approach is the path to real intelligence, and investors recently backed him with $1.03 billion.

> **AMI Labs**
>
> We are enabling the next AI revolution, by building a new breed of AI systems that (1) understand the real world, (2) have persistent memory, (3) can reason and plan, (4) are controllable and safe.
>
> Our main goal is to build intelligent systems that understand the real world.
>
> Real-world data is continuous, high-dimensional, and noisy, whether it is obtained through cameras or any other sensor modality.
>
> Generative architectures trained by self-supervised learning to predict the future have been astonishingly successful for language understanding and generation. But much of real-world sensor data is unpredictable, and generative approaches do not work well.
>
> AMI is developing world models that learn abstract representations of real-world sensor data, ignoring unpredictable details, and that make predictions in representation space.
>
> Action-conditioned world models allow agentic systems to predict the consequences of their actions, and to plan action sequences to accomplish a task, subject to safety guardrails.
>
> AMI will advance AI research and develop applications where reliability, controllability, and safety really matter, especially for industrial process control, automation, wearable devices, robotics, healthcare, and beyond.
>
> Founded by a globally distinguished team of scientists, engineers, and builders, AMI is a frontier AI research lab operating across three continents from day one.
>
> We share one belief: real intelligence does not start in language. It starts in the world.
>
> We can build the future of AI together: with industry partners, product developers, and with the global academic research community via open publications and open source.
>
> If this vision resonates with you, come join us:

AMI Launch Post

There are trade-offs to the latent approach, as there are trade-offs to generative approaches.

LeCun argues that the thing that *seems* like the biggest trade-off, a loss of fidelity in exchange for speed, is not actually a trade-off. His position is that the detail you lose is detail you *should* lose, that trying to predict every pixel is not just expensive but actively counterproductive — the model wastes capacity on inherently unpredictable visual details instead of learning the abstract causal structure that actually matters for reasoning and planning. Imagine if you needed to simulate every photon when you imagine catching a ball. Your brain might explode. There's some level of detail that is not "every single detail" that is optimal. LeCun's argument is that with World Models, the optimal level requires fewer details than many people, including us, think.

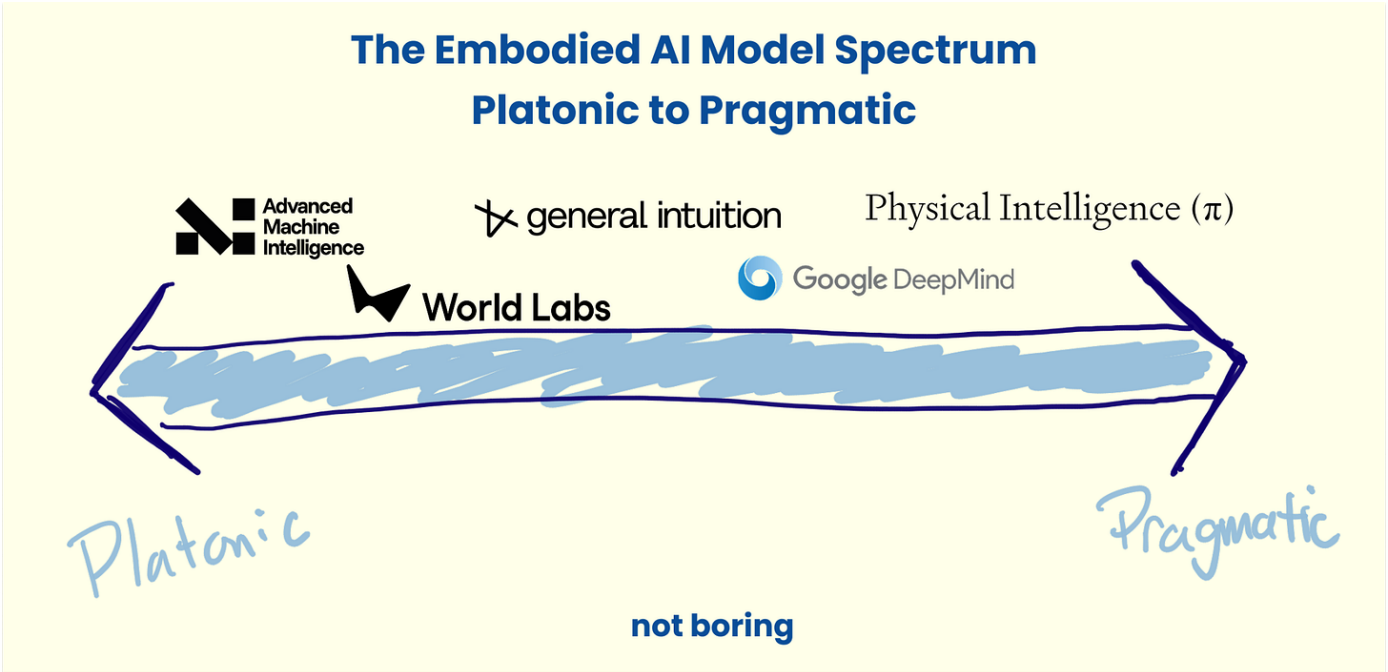
There *are* other trade-offs to keep in mind, however, that LeCun hasn't mentioned.

One is that latent models are trickier to evaluate. You can't look at the output and see if it makes sense intuitively the way you can with generated video, and they can't serve as training grounds for human-in-the-loop systems, because humans can't operate inside latent space. We need to see the world to act in it.

Another related downside is that your iteration speed slows down when you can't visualize predictions or interpret the loss. Humans are very good at noticing when something is visually off; we did not evolve to spot discrepancies in predicted latent encodings of the future ([0.13, -1.02, 0.44, 0.07, ...], MSE = 0.0187). And iteration speed is what matters most in modern ML, because modern ML progress mostly comes from empirical search, not from knowing the right design ahead of time.

Latent models are also more challenging to train for similar reasons. Additionally, the lack of strong supervision in the learning objective leads to collapse issues which require a bunch of tricks to fix. Why? The JEPA objective is to predict the encoding of the future based on the encoding of the past, but you can satisfy that objective with trivial encodings (e.g. set everything to 0, there is 0 loss), so we need to make sure representations don't collapse.

There is a spectrum in creating environments in which agents can train. On one side is what is practical today, and on the other is the platonic ideal. **Latent World Models are almost the opposite side of the Practical ←→ Platonic spectrum to VLAs,** which we will cover below**.**



They are closer to what researchers believe to be the technical platonic ideal, but they face real challenges in practice today. That said, new methods like LeJepa are closing the gap, and talent is flooding into the field.

Chris Manning, Ian Goodfellow, and Fan-Yun Sun have also joined the cohort of Latent World Models, starting latent lab Moonlake. Their entrance on the side of latent is notable. Manning helped pioneer neural natural language processing and co-created GloVe, which was the dominant word-embedding model before transformers. And Goodfellow invented **GANs (generative adversarial networks)**, which were the first widely successful way to train neural networks to generate realistic synthetic data.

In a recent X post, the Moonlake co-founders explained their approach to building efficient World Models. It is an interesting hybrid.

The plan is to generate full game environments to attract human players and collect action-labeled data. Afterwards, they model the world in semantic/symbolic space rather than pixels. As in, they use beautiful game environments to attract real human players, because they need humans to generate action-labeled data. But once they have that data, they discard the pixels entirely and train on abstract representations instead, betting that the underlying patterns matter more than the visual detail.

Ultimately, we don't view latent and general models as opposed to each other. Moonlake's hybrid approach is evidence of that. They just serve different goals. Latent World Models are generally more computationally efficient since they discard some information, with advantages for representation learning and planning. Generative World Models should be more general, since in theory they capture all visual information, with advantages for interpretability and generalization. Both can be used for many different purposes, including training agents with reinforcement learning.

Now, let's turn to Generative World Models.

Generative World Models

Generative World Models are the closest thing to simulating human-perceived reality that we're aware of. If our world is a simulation, it's probably a Generative World Model of some sort.

This is the paradigm we at General Intuition predominantly focus on to build the World Models in which our policies learn. It's also the one that recently blew the world's minds when Google DeepMind released Genie 3.

The video — and, Genie 3 itself, if you get a chance to play with it — gives you a felt sense for what makes Generative World Models different. They're *interactive.* They *respond.*

They generate human-observable, interactive futures that you can see, act in, and learn from. You can see what the model thinks will happen next. The model takes in a state and an action and produces a plausible next state, which you can act in again. Based on the updated state and new action, it produces the next plausible next state, and so on and so forth. A human can look at the output and say, "That's wrong, walls don't bend like that" or "Yes, that's exactly what happens when you turn a steering wheel at speed."

Generative World Models predict the observations themselves in pixels, video, or 3D scenes, allowing agents and humans to interact with the simulated environment. The dream is visible and playable.

This is what improves the training loop in many cases. Both generative and latent models can learn in imagination. Yet when visual details matter, or when the downstream task is not yet known, Generative World Model learning, with all its pixel-level detail, tends to outperform.

This only works if the generated environment is rich enough to learn from. The further the generated world is from reality, the worse the lessons the agent learns are. And the less successful it is when it goes back to real games. This is what DIAMOND showed, that when there is more detail in the generated world, agents are smarter.

At General Intuition, we are building on this diffusion and flow-matching architecture. It is developed, in part, by researchers who are now our co-founders and who built IRIS, DIAMOND, and GAIA-2.

Wayve, the birthplace of GAIA-1 and GAIA-2, is the leader in Generative World Models for autonomous driving. By using a large latent diffusion World Model offboard, they aim to dream up edge cases that would take millions of driving miles to find in reality, train driving policies on them, *score* the driving policies on their performance in simulation, then distill that dreamed experience into a smaller onboard policy that can reason through those same scenarios in real-time. The tweet below shows Wayve zero-shotting a drive on Japanese roads in the latest installment in a series doing this around the world.

Day 3, video #3. Every day this week we are releasing a new hour-long, zero-shot autonomous drive from a new country around the world. Do you recognise this location? It is Japan! 🇯🇵 Yokohama in wet conditions from highways to tight, bustling urban roads. Check it out!

Decart is applying Generative World Models to real-time generative simulation, producing playable worlds that respond to the users' actions. It's the playable version of the Generative Video Models or 3D Reconstruction Models. On the Oasis landing page, it calls the model a "video model," but follows up with this distinction: "Every step you take will reshape the environment around you in real-time."

Interestingly, Decart currently runs on Nvidia GPUs but plans to use Etched Sohu chips. Etched chips are custom ASICs designed to run transformers and would allow Decart to improve latency and run continuous inference, both of which are much more important when generating responsive worlds in real-time than when generating a video or 3D rendering upfront.

Runway, too, is blurring the lines between video generation and world generation, as mentioned in the Video Model section. During its Research Demo Day 2025, Runway co-founder and CTO Anastasis Germanidis explained the company's evolution that started from *"generative AI models [as] viable tools for creative expression".* They then evolved towards World Models (while still making incredible progress in video models.)

"To build a World Model," Germanidis explained, "We first needed to build a really great video model. We believe that's the right path to building World Models, that teaching models to predict pixels directly is the best way to achieve general purpose simulation."

Google DeepMind took a similar approach; Genie 3 was built on top of Veo.

These World Models are extremely important. But remember that they are only half of the equation. From the very start, whether it was Schmidhuber in 1990, Sutton with Dyna in 1991, the plan was to use World Models to train Agents to act inside of the world, and then to transfer those learnings out into the real world.

Embodied Agents

We want to share a few of the main Embodied Agent examples out there today, and their respective approaches: **Physical Intelligence and other robotics companies' VLAs** (**Vision-Language-Action Models), DreamerV4's a Latent World Model Agent, Google Deepmind's Sima2 General Embodied Agent, and General Intuition's General Agent approach.**

Physical Intelligence - Vision-Language-Action Models (VLAs)

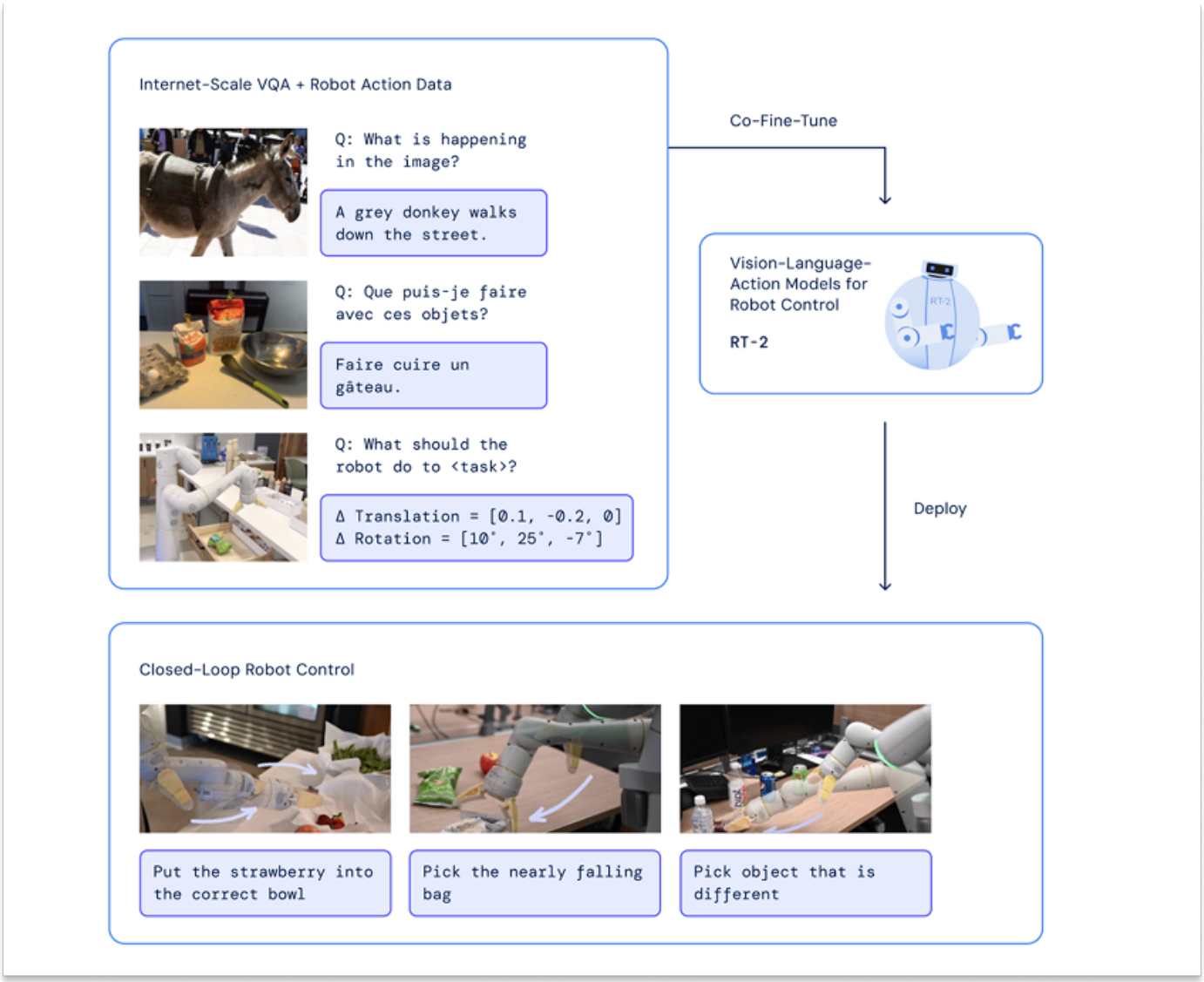
Modern multimodal LLMs come with a model called a **VLM, or Vision-Language Model**, a model that can see and read. Feed it an image and a question, like "What objects are on the table?" or "Is this door open or closed?", and it produces a coherent, grounded answer.

GPT-5, Gemini, and Claude are all VLMs in this sense; they can see and reason. When you send it a picture of a mountain and ask it to geolocate it, it's using its VLM. VLMs are also the perceptual and reasoning backbone of most modern Agent systems designed to operate in physical or interactive environments, like PaLM-E or SpatialVLM.

VLMs are not exactly Agents, but they are core components of most of them. We bring them up because a VLA is a VLM that has learned to act, and it is the pragmatist's answer to the Agent problem.

In 2023, Google DeepMind published a paper called *RT-2: Vision-Language-Action Models: Transfer Web Knowledge to Robotic Control* to propose a solution.

Take a VLM that understands a scene and what to do in it, and then bolt on an **action head** that translates human language instructions into instructions that the robot understands, like to change a position or rotate.



Google DeepMind, RT-2

Since then, VLAs have become the dominant paradigm in robotics, and they've worked surprisingly well.

Every other paradigm we're discussing says something like: "Images, videos, spaces, and actions are fundamentally different than words. We need to train and architect the models that generate them differently than the models that generate words."

Vision-Language-Action Models (VLAs) say: "That may be true! Those approaches might be platonically better. But that doesn't matter in practice, because the vision-language model infrastructure and data are so far ahead."

In his not boring primer on robots, Standard Bots' Evan Beard wrote a thorough explanation of VLAs for robotics that included what he called a "spicy take":

> ***We're not working with language-model infrastructure because it's the perfect architecture for robotics.*** *It's because we, as a species, have poured trillions of dollars and countless engineering hours into building LLM infrastructure. It's incredibly tempting to reuse that machine.*

> *So, despite its imperfections, taking an LLM and sticking on an action head to predict robot motions (all together known as a VLA) is the best way for us to train the base models that learn many skills from demonstrations across many different customers and tasks.*

It's pretty ingenious. Of course, there are challenges to this approach, which Evan highlighted:

- *Robotics success so far has leaned heavily on diffusion-style control*
- *LLMs are autoregressive and token-based, with less room for error*
- *Physical actions don't map cleanly to tokens*

Additionally, compared to World Models, VLAs require collecting a large amount of real-world robotics data; they don't seem to generalize out-of-distribution particularly well.

That said, Physical Intelligence, known as π or Pi, has gotten incredibly far with its VLA bet.

Pi's first generalist policy, π_0: Our First Generalist Policy, inherits semantic knowledge and visual understanding from internet-scale pretraining and trains on data from seven different robotic platforms across 68 unique tasks, including folding laundry, bussing dishes, routing cables, assembling boxes, and packing groceries, all of which require dexterity in the real world on real hardware. Their follow-up, π_0.5: a VLA with Open-World Generalization, performs better in new environments like cleaning up a kitchen or bedroom in a home the model has never seen before.

OK, but can it actually learn and get better over time as it works and makes mistakes in the real world?

November 2025's π*0.6: a VLA that Learns from Experience suggests that it's possible, with demonstrations in tasks like making espresso, folding boxes, and folding laundry.

But those are simple, repetitive tasks. Most of what the robot sees is in distribution. Can it actually do more complex, multi-step tasks that take a long time to complete?

Earlier this month, Pi released VLAs with Long and Short-Term Memory and showed that robots using MEM (Multi-scale Embodied Memory) can clean up an entire kitchen, set up the ingredients for a recipe, and grill a grilled cheese sandwich. They can also learn from their mistakes.

A robot tries to pick up a chopstick or open a refrigerator door. Without memory, it fails the same way repeatedly. Each attempt is a clean slate with no knowledge of what just went wrong. With memory, it tries a different approach after the first failure. And it succeeds.

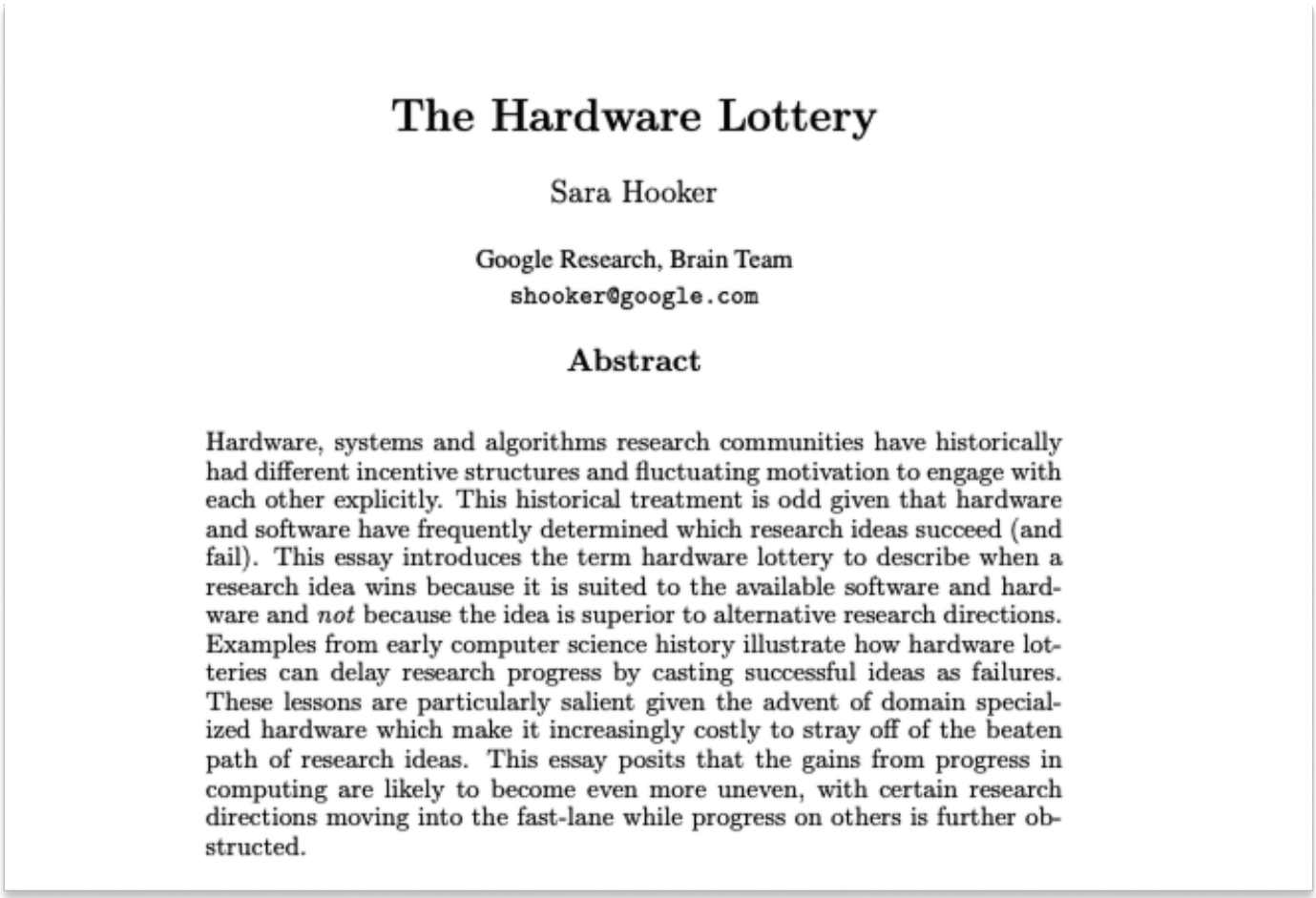
MEM doesn't change the underlying architecture, which is still sub-optimal for embodied systems. Most of the parameters still live in the language backbone. The action head is still downstream of reasoning. But Physical Intelligence's existence raises a fascinating question. **Do these architectural limitations actually matter in practice?**

If Latent World Models are on one side of the Platonic ←→ Pragmatic Spectrum, VLAs are on the other.

To date, Pi has been able to engineer their way around architectural limitations to make increasingly capable robots. Their progress is not slowing down. It seems to be accelerating.

Theirs is a bet with historical precedent. The ideal technology — the solution that is technically superior — does not always win. This is the key takeaway from W. Brian Arthur's 1989 paper, *Competing Technologies, Increasing Returns, and Lock-In by Historical Events*. Markets often converge on the technology that gets adopted first, because adoption creates increasing returns: better early product means more users and more capital, which means better data, more internal talent, and more developers, which means better products which means more users and capital, and so on.

This is also the point of Sara Hooker's 2020 paper, *The Hardware Lottery:* "This essay introduces the term hardware lottery to describe when a research idea wins because it is suited to the available software and hardware and not because the idea is superior to alternative research directions."



The Hardware Lottery

Sara Hooker

Google Research, Brain Team
shooker@google.com

Abstract

Hardware, systems and algorithms research communities have historically had different incentive structures and fluctuating motivation to engage with each other explicitly. This historical treatment is odd given that hardware and software have frequently determined which research ideas succeed (and fail). This essay introduces the term hardware lottery to describe when a research idea wins because it is suited to the available software and hardware and *not* because the idea is superior to alternative research directions. Examples from early computer science history illustrate how hardware lotteries can delay research progress by casting successful ideas as failures. These lessons are particularly salient given the advent of domain specialized hardware which make it increasingly costly to stray off of the beaten path of research ideas. This essay posits that the gains from progress in computing are likely to become even more uneven, with certain research directions moving into the fast-lane while progress on others is further obstructed.

From the outside, it seems that Pi's strategy is to ride the transformer architecture's increasing returns and trying to generate its own, to create path dependence with VLAs before World Model-specific architectures gain traction, in an attempt to win its own hardware lottery.

They're not the only company making this bet. Skild, the closest direct competitor, is building on VLAs. A number of robotics companies incorporate VLAs and VLMs in one way or another. And now, it looks as if the approach is spreading to the whole factory.

Recently, the WSJ reported that former OpenAI Chief Research Officer Bob McGrew is raising $70 million at a 700 million valuation for his new startup, Arda, in a round led by Founders Fund and Accel, with participation from Khosla and XYZ. Details are light, but the WSJ's description sounds like it's going to at least involve VLMs and VLAs in some way: "Arda is developing an AI and software platform, including a video model that can analyze footage from factory floors and use it to train robots to run factories autonomously."

The more well-funded and talented companies that move in this direction, the more deeply grooved the path becomes.

Personally, I don't think VLAs and World Models are really competing. They're trying to reach acting in the physical world from different directions. VLAs are language first, while World Models are video actions-first. My guess is that they'll converge and both be part of the solution.

Dreamer V4 - Latent World Model Agents

Latent World Model Agents are Agents trained inside of Latent World Models. The latent approach has a natural elegance for Agent training specifically.

Because a Latent World Model operates in compressed abstract space, the Agent's planning and policy learning can happen very efficiently, with no pixel generation required. The agent basically practices by thinking, in the same way a chess grandmaster runs through variations in their head without moving the pieces, or the way a lucid dreamer trains inside the dream.

The canonical example is Dreamer, from Danijar Hafner, now at Google DeepMind. Dreamer's insight is elegant: if you have a good enough Latent World Model, you don't need to touch the real environment during training at all. The Agent imagines sequences of actions and their consequences entirely in latent space, receives a reward signal, and updates its policy, all without a single real-world interaction. When it finally goes into the real environment, it already knows what to do.

Dreamer has achieved remarkable results across a wide range of tasks, from games to continuous control to robotics, all from this purely imagined training. It is the research proof of concept that World Model training works, that an Agent can learn to act in the real world by dreaming. It seems as if Hafner is taking his research proof commercial. Earlier this month, *The Information* reported that he and Wilson Yan are raising $100 million to build a World Model company in this paradigm called Embo, which suggests they're going after embodied systems.

The challenge, as with Latent World Models generally, is that the Agent's learned behavior is only as good as the latent representation. If the World Model's abstract encoding misses something causally important, like the exact texture of the floor that determines whether the robot slips or the precise angle of an object that determines whether it can be grasped, the Agent won't know to care about it, because the model didn't encode it. Garbage in, garbage out, but the garbage is invisible.

Moonlake's hybrid approach, which we discussed earlier, is an attempt to thread this needle: attract humans with beautiful generative environments to collect action-labeled data, then discard the pixels and train the Agent in abstract space. Use the generative world to get the data. Use the latent world to do the learning. It's an interesting bet that the two approaches are more complementary than competing, and it may prove correct.

Notably, we haven't yet seen the JEPA Agents. JEPA is a World Model architecture, not an Agent architecture, but we expect AMI Labs will close this loop. AMI is still building its World Model, and the Agents that train inside it haven't yet been publicly demonstrated, but we're watching closely.

General Embodied Agents

SIMA2 - Generalist Embodied Agents from VLM Backbone

In November 2025, Google DeepMind released SIMA 2: An Agent That Plays, Reasons, and Learns with You in Virtual Worlds.

SIMA 2 combines a Gemini backbone with a World Model trained on 3D game environments, giving the Agent an understanding of language that allows it to receive and reason about goals, as well as the spatial-temporal understanding to execute on them. In this architecture, Gemini fills the role that we mentioned VLMs play in our system.

What makes this a different paradigm from VLAs is the direction of citizenship. In a VLA, language is first class, images are second class. Beyond the ordering of modalities, there is also the training data, mostly static images interleaved with text. In an Agent equipped with a World Model, video is first class, actions are introduced from the beginning, and the training data is directly aligned with the downstream behaviors we're looking for. The Agent's fundamental competence is spatial-temporal. If you tell it what it needs to do, it knows how to move through the world to do it.

SIMA 2 can play games on its own. It can learn, reason, and improve. The more it plays, the better it becomes, not just in the games it's played, but in *any* game it plays. It is even able to play in whichever *generated* world it gets thrown into, even when it's never seen it before. This, Google DeepMind believes, is a "step towards creating AI that can help with any task anywhere, including one day in the real world."

Google DeepMind has put out an extensive amount of research. They have pushed World Models and embodied AI forward on multiple fronts. They coined the term "VLA." They released Genie 3. They developed SIMA 2. The way they trained AlphaGo, letting the Agent play against itself over and over and over again, informed how World Models are trained to this day.

General Intuition - Generalist Agents From Actions & World Models

> Similarly to Google DeepMind, we also believe that Generalist Agents will play a major role in how embodied systems operate to do useful things.

First, create the dream. Then, let Agents run around inside of it. Let them play and mess up and learn and win. Then, transfer those learnings into other dreams, and even into the real world.

Recall the Matrix. When Neo needed to learn Kung Fu, he plugged into a virtual dojo where he trained against Morpheus in a training environment superior to the "real world." After that? "I know Kung Fu." World Models are the virtual dojo. Neo is the Agent.

I Know Kung Fu

This is the question that Ha and Schmidhuber asked eight years ago: **can Agents learn inside of their own dreams?**

In a remarkably short amount of time, the space has come to an answer: **yes.**

Yes… If You Have Action-Labeled Data (Or Can Get It)

Today, I want to share a little bit more about our approach and the results we're starting to see.

Every approach I've written about so far runs into the same wall, eventually: It needs better data. Video is abundant, but it lacks depth. It has no action labels. And without knowing what actions caused what we're seeing, video data is like shadows, the shadows on Plato's cave wall.

And Yann may be right that you can *infer* action, but anyone using inferred action has separate scaling laws to consider: inferring the actions themselves. Inferring actions takes compute, time, and attention away from doing the things you can do once you understand actions, and while inferred actions might look good on benchmarks, they struggle deeply on edge cases. Even well-inferred actions are approximations of what someone actually did: some things just aren't visible in video, like moving the rudder on a plane landing from the cockpit.

Hint: if you don't do it, you crash. That's why ground truth is crucial.

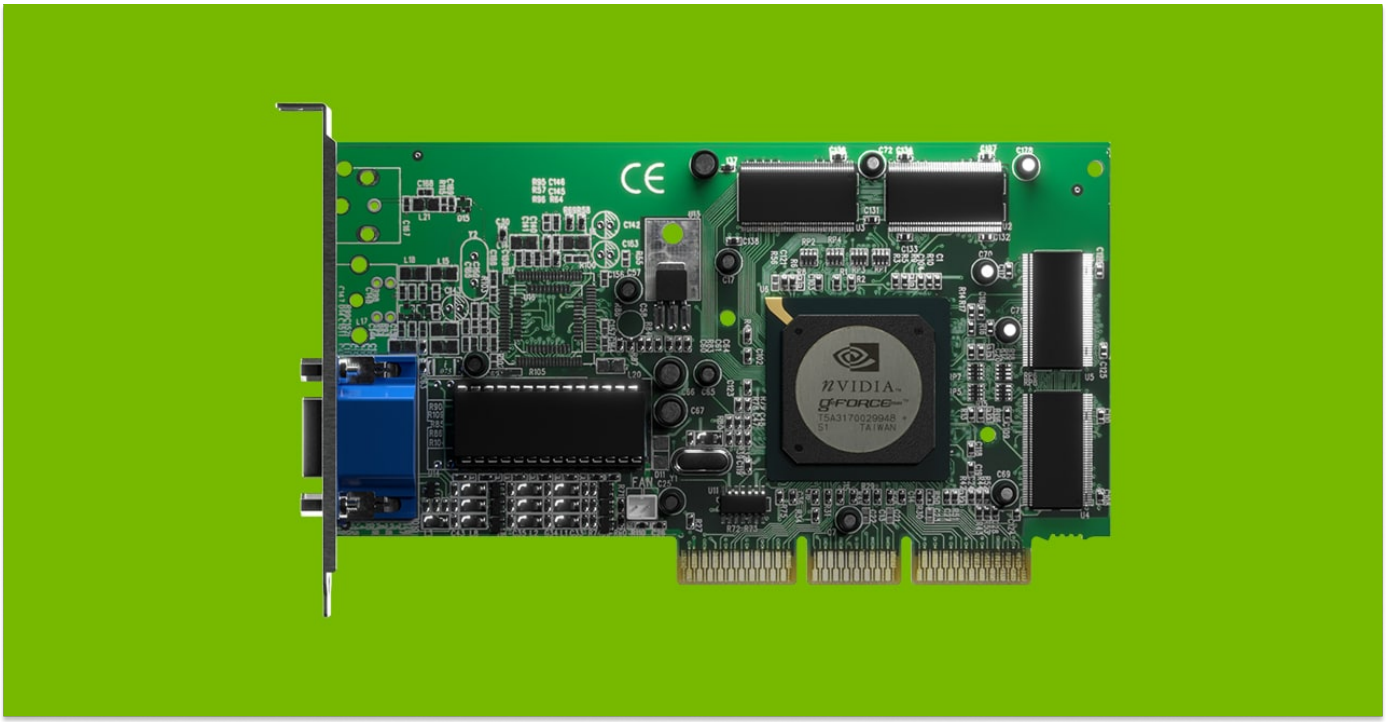
You need to find a way to get action-labeled data. The closer to ground truth, the better. Luckily, we have a fantastic starting point, thanks to Medal.

Before there was General Intuition, there was Medal

Earlier, we talked about the importance of games in the development of AI. AlphaGo. Deep Blue. These are *intentional* uses of games in AI.

There is an even richer history of *accidental* links between games and AI, lucky breaks.

Nvidia is the example you likely know. Jensen founded Nvidia to make chips for real-time graphics in games in 1993. Six years later, in 1999, Nvidia released its first "Graphics Processing Unit" (GPUs), the GeForce 256.



The world's first GPU: NVIDIA GeForce 256, 1999

A few years later, around 2005, researchers began experimenting with GPUs for neural nets. In 2007, Nvidia released CUDA, to make ML on GPUs practical. In 2009, three Stanford researchers — Rajat Raina, Anand Madhaven, and Andrew Ng — showed that GPUs could accelerate deep learning by 70-100x for unsupervised learning.

Three years later, in 2012, the AlexNet team7 decimated their ImageNet competition using GPUs. Within a year, everyone in deep learning had switched to GPUs. "Everyone in deep learning" was still a small community at the time, but by then, Bitcoin miners had already been using GPUs. They were 50-100x more efficient for Bitcoin's SHA-256 hashing than CPUs.

They soon switched to ASICs, but in 2015, Vitalik Buterin and his team released Ethereum, whose memory-heavy workloads were harder to optimize with ASICs. Ethereum mining ran on GPUs from 2015, through the GPU shortage it caused during the 2020-2022 crypto boom, right up until Ethereum switched from Proof-of-Work to Proof-of-Stake and left a GPU glut in its wake. Crypto tanked anyway, and in the same month crypto peaked, Nvidia's stock did, too, tumbling 66% over the next year, until OpenAI released ChatGPT and, since then, Nvidia's market cap has grown 10x to the $4.4 trillion behemoth we know today.

Google Finance as of 3/9/2026

I mean, who could have predicted all of that?

When I taught myself to reverse engineer things, and learned how to code to build a private Runescape server when I was 13, I couldn't have predicted that it would lead me to where I am today, either. Reverse engineering is the ultimate form of deductive reasoning, and spending a lot of time doing it as a kid is very good for your brain. This then lends itself well to figuring out complex systems in a rapidly changing world.

Runescape developers took the wilderness and free trade out of the game. I wanted to put it back, so I learned how to reverse engineer. The business that grew out of that did well for a teenager — we were making ~$1.5 million per year by the time I had to shut it down in 2015 at age 18, when I became an adult and would have been liable for the stuff I built. But I'd made enough money for my age that I could do whatever I was passionate about. I joined Doctors Without Borders (MSF) at 19 years old, and stayed for three years to work on Ebola & Humanitarian Mapping. I spent some time at Google Crisis Response before the gaming itch got me again.

At the time, we worked in London, very close to the DeepMind team. It was 2014 and I did not think it was that interesting or that likely to work. Demis deserves so much credit and respect for his vision. Few understand how hard it was for them to get here.

Working on Ebola with MSF at 19. We almost got the Google London office evacuated that day, and had to label the PPEs with "fake test" from thereon out

In 2018, I teamed up with my previous colleagues from building RuneScape servers. We built a game called Get Wrecked, which got a lot of signups. But it lost players quickly because we didn't have enough player liquidity; it was a competitive game, and we needed to have enough people of all skill levels that people would always be able to find someone at their level to play against, which is very hard to bootstrap. To fix that problem, we built a way for people to watch game clips on the platform. A couple of times a day, we'd send a push notification that the game was live to get enough players playing at once.

The clips platform, Medal, went viral on the Rocket League subreddit. It was getting so many downloads that it became almost immediately clear that that was the bigger opportunity. We decided to focus on Medal.

We never ended up releasing the game. Medal just kept growing. Today, players from around the world upload 1B+ gaming clips to Medal each year.

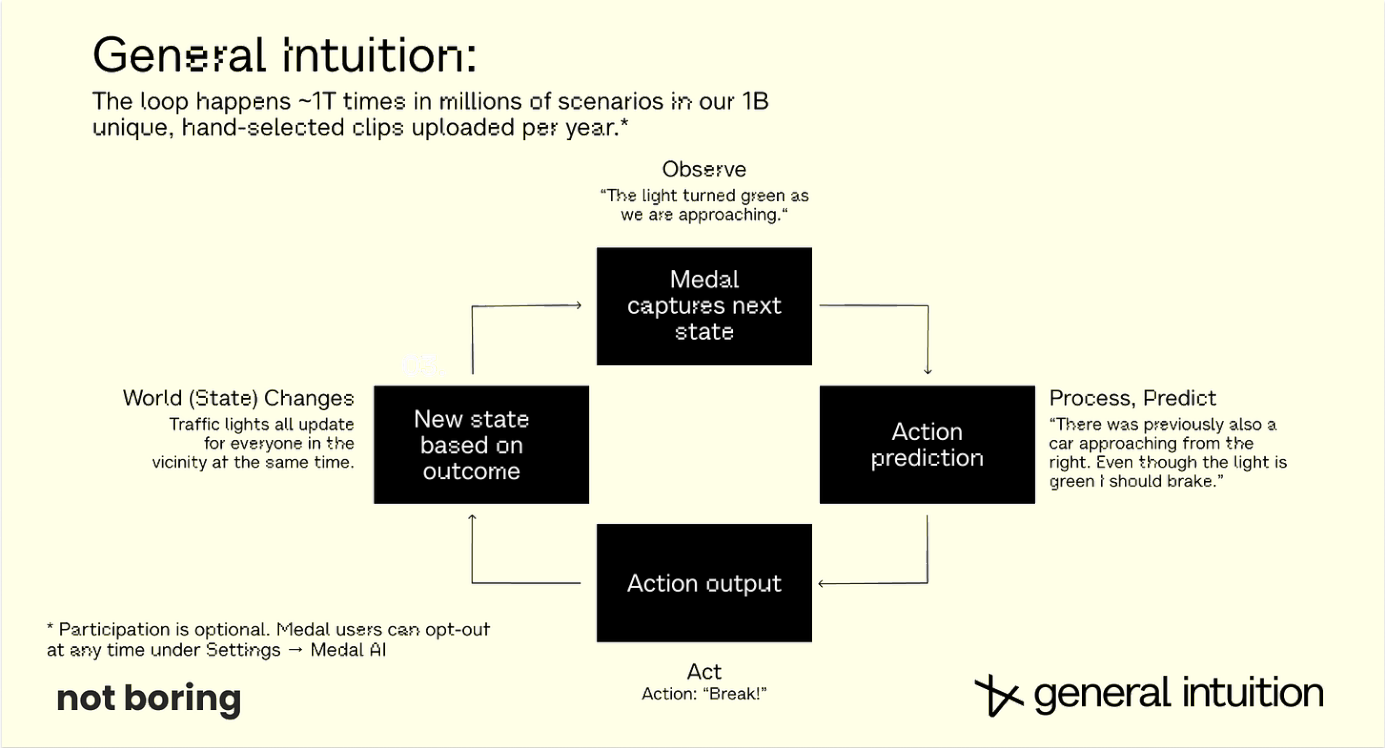
We couldn't have planned a better dataset with which to build World Models and policies.

Medal's upload volume puts it on par with YouTube. Gamers upload millions of clips per day, across tens of thousands of environments, already hand-selected by players for highlights and adverse events. In other words, they share the content they think is worth sharing: their best performance, wildest encounters, closest calls.

Medal data has something that YouTube data does not. It comes enriched with metadata from our social network (views, likes, comments) and most importantly, **in-game actions.** We only record the game actions on the local machine, only storing the in-game action names (e.g. Move Forward) and never the keys that were pressed to achieve that action. More than data, this has enabled us to ship Medal's most requested feature, keyboard and controller overlays. These overlays let our gamers showcase the precise actions they took behind every amazing moment.

Each clip has exactly what the player saw, next to the exact player actions that followed, using many of the same systems we use to control robots today. Frames from games also have the benefit of being information-complete. Unlike real world video, where you have to account for pose estimation (estimating what the human sees, which in itself is a lossy process) — you may see things the camera does not in the real world — but not in games. What is recorded and what you see is always identical, which we think makes it better training material.

This gives us trillions of examples of players running the loop of observe, predict, and act. This is the foundation of intelligence, and there is no loss of information throughout.

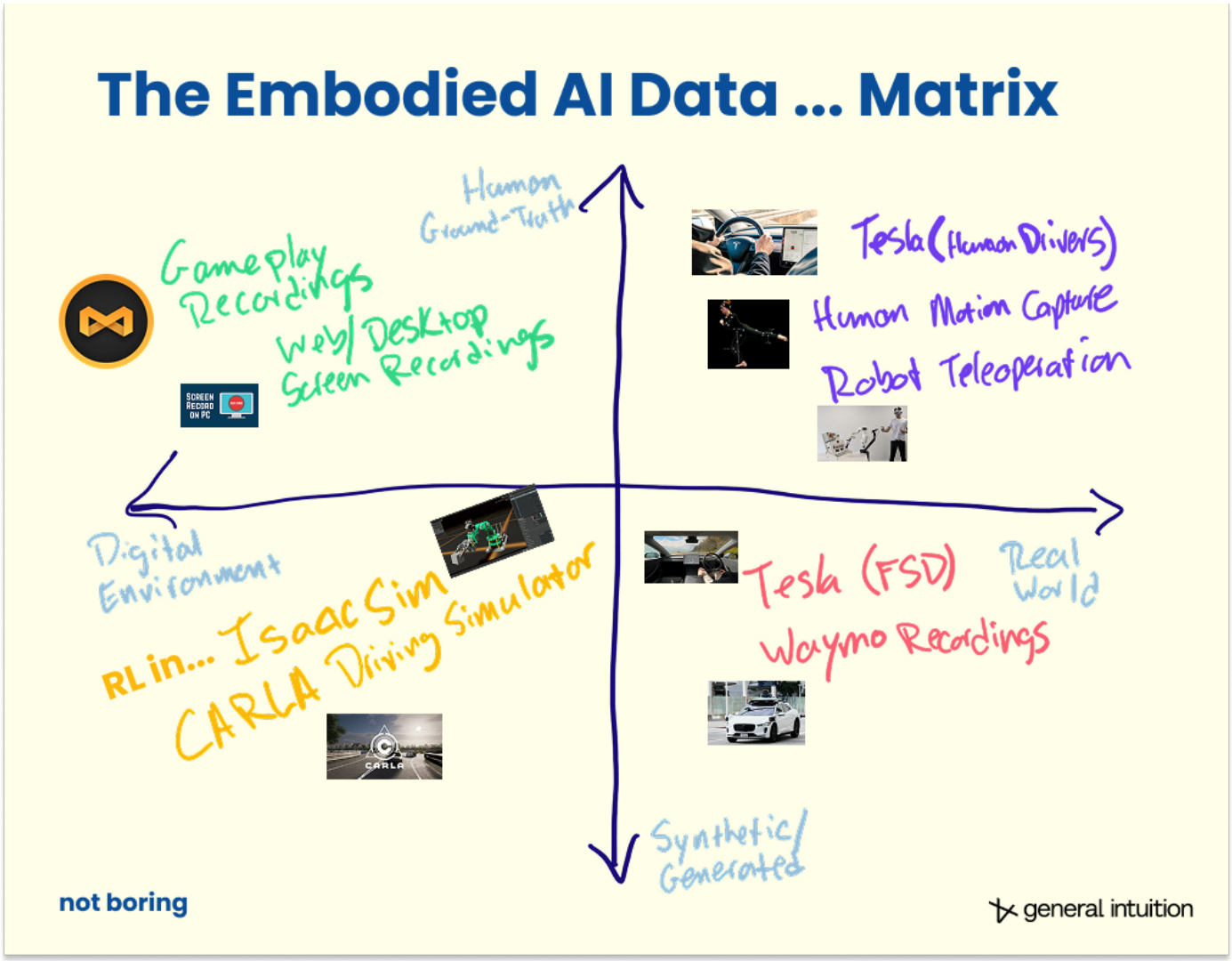


On Data

To understand what we're doing, you need to grok the difference between game data and synthetic data.

The confusion is that people associate "digital" with "synthetic," but the real distinction isn't the environment in which the data was generated, but the data itself.

There can be synthetic (i.e., generated) data created in the physical world, like in the human-constructed environments Boston Dynamics and other robotics companies train their robots in, just as there can be human ground truth data in the digital world. Data breaks down into a quadrant like this.



What makes our gaming data "ground truth human data in a digital environment" is that what we are capturing is real human responses that observe → predict → act loop.

The closest comparison to our approach is GitHub data. It captured the history of human engineers' coding and was used to train machines that can code better than humans. The question is whether that same idea works outside of the computer. We believe (and are seeing signs) that learning from gaming data transfers to the physical world.

Games turn out to be the perfect training ground for learning intelligence. They contain thousands of simulated worlds with physics, strategy, cooperation, text, interface use, competition, and long-horizon planning. They are complex enough to require intuition, but structured enough to learn from at a massive scale.

Physical-world data alone cannot reach the diversity or scale required to learn general intelligence. LLMs lack data on dynamics and atoms. But games act as the ideal intermediary: a bridge between the digital world of bits and the physical world of atoms.

Still, there is a threat to the ground truth stance. As mentioned earlier regarding Yann LeCun's take, every video is action-labeled data if you're good enough at inferring action. While this may be true in the long run, it's also probably extremely impractical today. That's why you gotta love Yann — nobody else would think to do it this way. Yann and I talked about this dilemma in Paris in December if you want to go deeper into the weeds.

General Intuition@gen_intuition

Yann LeCun (Chief AI Scientist, Meta, @ylecun), @PimDeWitte (CEO, General Intuition), and Aude Durand (Kyutai, @aude_drn), talk about world models, embodied agents, Yann's new company, and the limitations of LLMs 0:00 - Introduction to World Models 5:00 - Why World Models,

Everything is a trade-off, right?

The optimal path forward is likely somewhere between where VLAs are today — the most practical but least elegant solution — and where AMI might be one day if everything goes well. It all comes down to your approach to data.

Data is *the* problem for any company that wants to solve embodied AI. Evan and Packy wrote about it in *Many Small Steps for Robots*, and it is the thing we are focused on at GI.

We believe our dataset is the most elegant answer to the data problem for general models. It is one that paves a path towards a general intelligence that feels familiar, the same way Tesla FSD feels like a familiar driver, but scales far beyond games or driving.

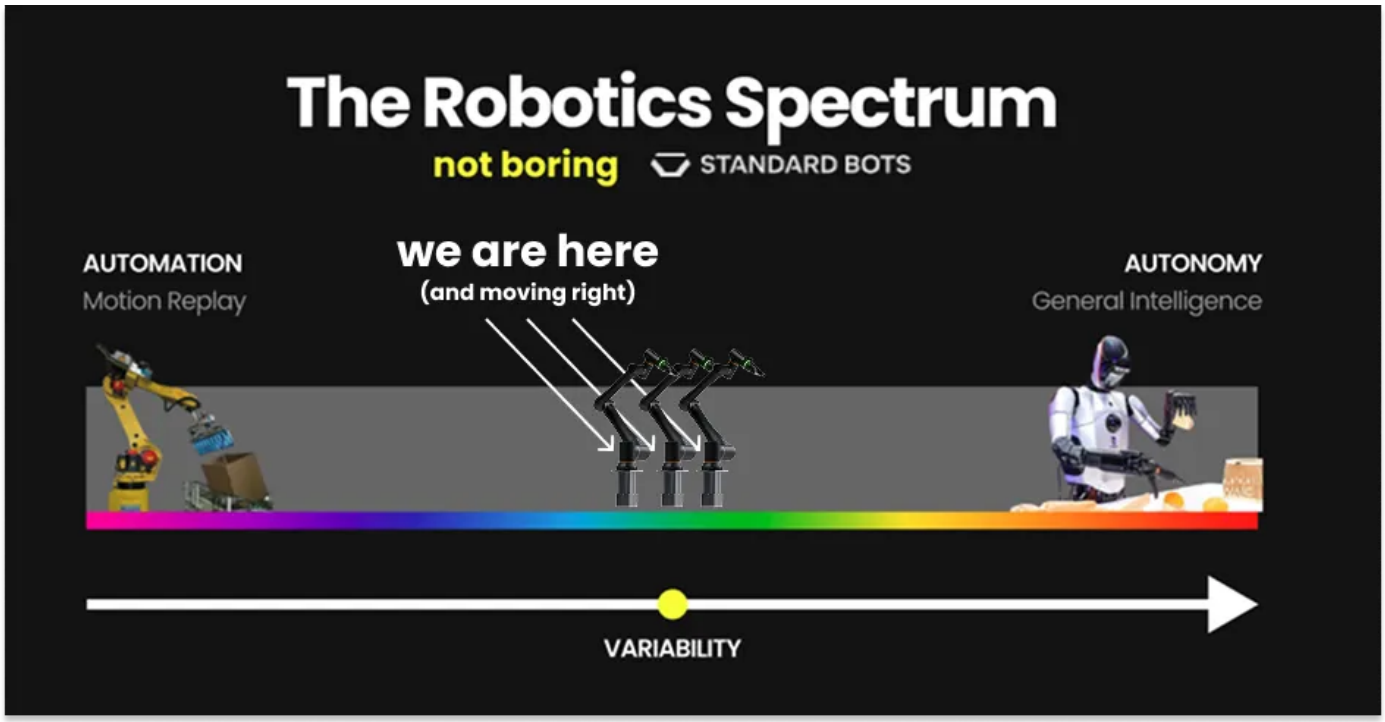
For general models, models that can power embodied AI intuitively and spontaneously in almost any imaginable real-world situation, the question is not simply how much data you can get.

Before throwing data at the problem, you need to understand your **transfer curves**.

Small Steps, Giant Leaps, and Transfer Curves

In their Robot essay, Packy and Evan wrote that there are two approaches to building economically viable embodied AI: Small Step or Giant Leap.

Evan and his company Standard Bots are pursuing the small step approach: getting paid to learn in the field, one use case at a time. They are collecting real-world data for a growing number of economically viable use cases across a wide variety of domains.



Their strategy is a fascinating one. By getting customers across many different industries and tasks to pay them to deploy, they collect diverse real-world data across wide distributions. Instead of hoping that more data in a narrow domain will generalize to tasks outside of the distribution, their goal is to put a ton of useful tasks *in-distribution* by going broad in the real world, not deep in one niche.

General Intuition and Standard Bots are coming at the same problem from opposite sides of the spectrum. General Intuition is attempting to solve generalization from the **digital** side: our bet is that gaming data will lead to broad priors about physics and actions. Standard Bots is attempting to solve generalization from the **physical** side: their bet is that real-world deployments will lead to broad priors about manipulation and industrial tasks.

These are complementary approaches to the data diversity problem. There's potential for a GI World Model to be the starting point for Standard Bots' post-training. We provide a base model trained on observed data in digital environments, which is scalable and economical to collect, and they post-train using the use case-specific data they get paid to collect to bring those use cases in-distribution and get to many 9s of reliability more quickly.

The approach that we find more challenging is the one that general models seem to be taking, which is collecting a ton of data and hoping it generalizes to out-of-distribution tasks. **General models need too much data across too many situations to collect it all by paying people to demonstrate tasks.**

Additionally, more data in the same domain doesn't automatically teach a model to handle situations it's never seen before. **At pre-training, not all data is created equal**, and I have not met someone building a general model for robotics who has been able to point me to the scaling laws that show that they can solve out-of-distribution use cases (things they weren't trained on) simply by adding more data. More data in a narrow domain does not automatically buy you generalization to a new one.

The scaling laws don't exist.

There are, as best we can tell, three distinct transfer curves that govern whether a World Model generalizes to new physical environments. They are not well understood — we are just starting to understand them. We can, however, give them names: **input modality transfer, sensor transfer, and environment transfer.**

The first is **input modality transfer**: How well does a policy generalize across degrees of freedom in the physical system it's controlling? This curve is steep for a humanoid robot with somewhere between twenty and sixty degrees of freedom. Each is continuous and often mechanically dependent on the others, and this curve is steep. A finger movement is not independent of the arm. Training on a game controller and expecting it to transfer cleanly to a twenty-DOF humanoid hand is, in research terms, a bet without scaling laws to back it up.

The second is **sensor transfer**: If the workload requires specialized physical sensors (tactile feedback, proprioception, depth), there is a separate scaling law for how much of that sensor-specific data you need before the model can reliably reason about it. Tesla explicitly worked on this problem. They spent years figuring out exactly how much LiDAR data they needed before they could entirely drop the chips. Most robotics companies are working on it implicitly, hoping the answer reveals itself in deployment.

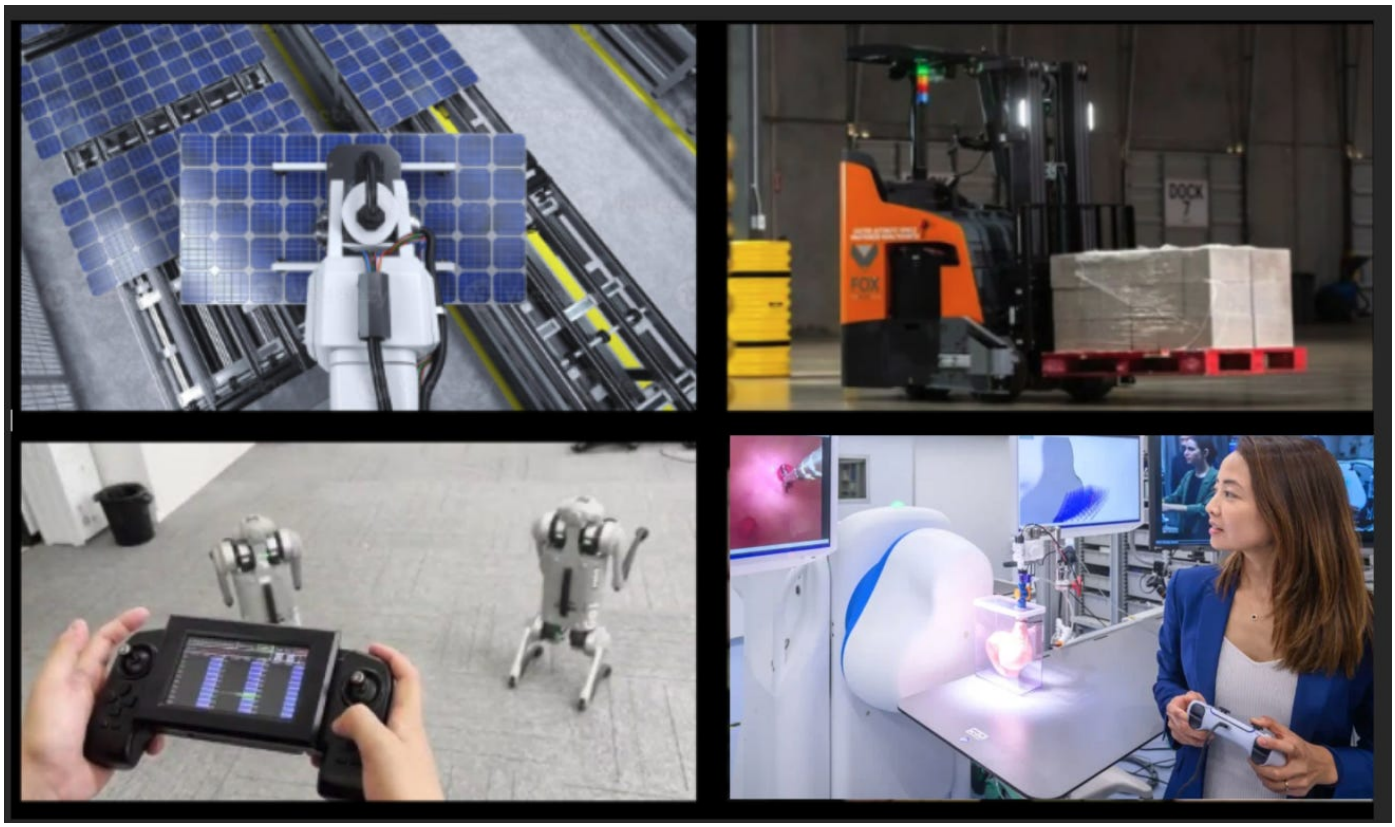
The third is **environment transfer**: How does performance degrade as the environment gets more complex, more stochastic, or more populated? Predicting the right action in a sports stadium with a thousand people around you is a fundamentally harder problem than predicting it on an empty field.

As we explained earlier, complexity doesn't scale linearly.

These three curves interact. Until you can map them, you can't know how much data of which type you actually need, which means you cannot justify the capital expense of going to collect it at scale. Companies that are collecting a hundred thousand hours of physical data today may find that a good World Model only needed ten thousand, or that they did need a hundred thousand, but ninety thousand of the hours they got were in the wrong distribution entirely.

Our bet, certainly conditioned on our starting position, is to collapse the problem.

By **focusing on game controller inputs**, we reduce input modality transfer to a curve we have already solved. We know we have enough data for game controllers, because we have billions of clips of humans using them. That eliminates one unknown. By focusing on vision-based inputs rather than specialized sensors, we eliminate the second.



Almost every physical system comes with a game-controller-like input modality, which includes steering wheels, keyboard and mouse and actual game controllers. Most are straightforward. Even humanoids ship with them. The challenge is just that if the degrees of freedom exceed what the controller can do, transfer is worse. So humanoids are further down our roadmap, but we see no physical limit to suggest that we couldn't build around the interface limit.

In short: If you can control almost any physical system with a game controller, and we have more data than anyone else in the world of what happens when a player uses a controller to take action, our Agents should be able to control almost any physical system.

The only remaining question is about **environment transfer**: can Agents trained inside of the dream operate in reality?

The Superhuman Future of the World (Models)

It has been a wild few weeks at General Intuition's offices in New York and Geneva. Everything that we've written about here has been working better than we expected. Like others, we're gaining conviction that Agents trained inside of the dream can operate within reality.

Why do World Models transfer?

The observe-predict-act loop is an abstraction for how causally structured systems work in general. Once a world model has seen N variations of the world via a diverse set of games, it only takes a small amount of fine-tuning to understand the dynamics of the N+1 variant that corresponds to the real world.

World Models learn to model the cause-and-effect of reality. If this cause-and-effect is understood at a fundamental enough level, this should enable World Models to generalize to new scenarios.

What might that mean? What are the implications of World Models that generalize?

Our goal is to enable embodied AI to understand the world, with our models controlling machines in any environment, including the real world. We aim to deliver a breakthrough moment for robotics, where out of nowhere, the progress is obvious and the models are easy to use.

That breakthrough won't look like the breakthroughs in LLMs, which went mainstream when they started talking to us like humans. We don't want machines that simply do what humans do. In fact, the point of machines is to do things that humans *can't*, to give us superpowers.

Robots don't need to look like us to work for us. Humanoids as a form factor were largely chosen due to the assumption that they had the most data to learn from on the internet, because so many of humanity's videos feature humans. If you don't need those videos, if you can learn directly from the actions in video games across embodiments, and need a lot less to transfer to reality, that assumption doesn't hold. We believe the future of robotics should be shaped by simpler, cheaper systems: machines with only the degrees of freedom that match the actual jobs to be done.

The human body is an incredible general-purpose platform, but it's rarely the optimal (or most cost-effective) form for any specific task. Instead of copying our anatomy, we should mirror the interfaces we already use instinctively: joysticks, wheels, gamepads, and keyboards. These tools are the product of decades of iteration, compressing human intent into a clean, universal action space, much like language does for thought. Robots can learn from the actions transmitted through these interfaces and specialize around them in a very general way, making broad deployment far more practical than chasing full human embodiment.

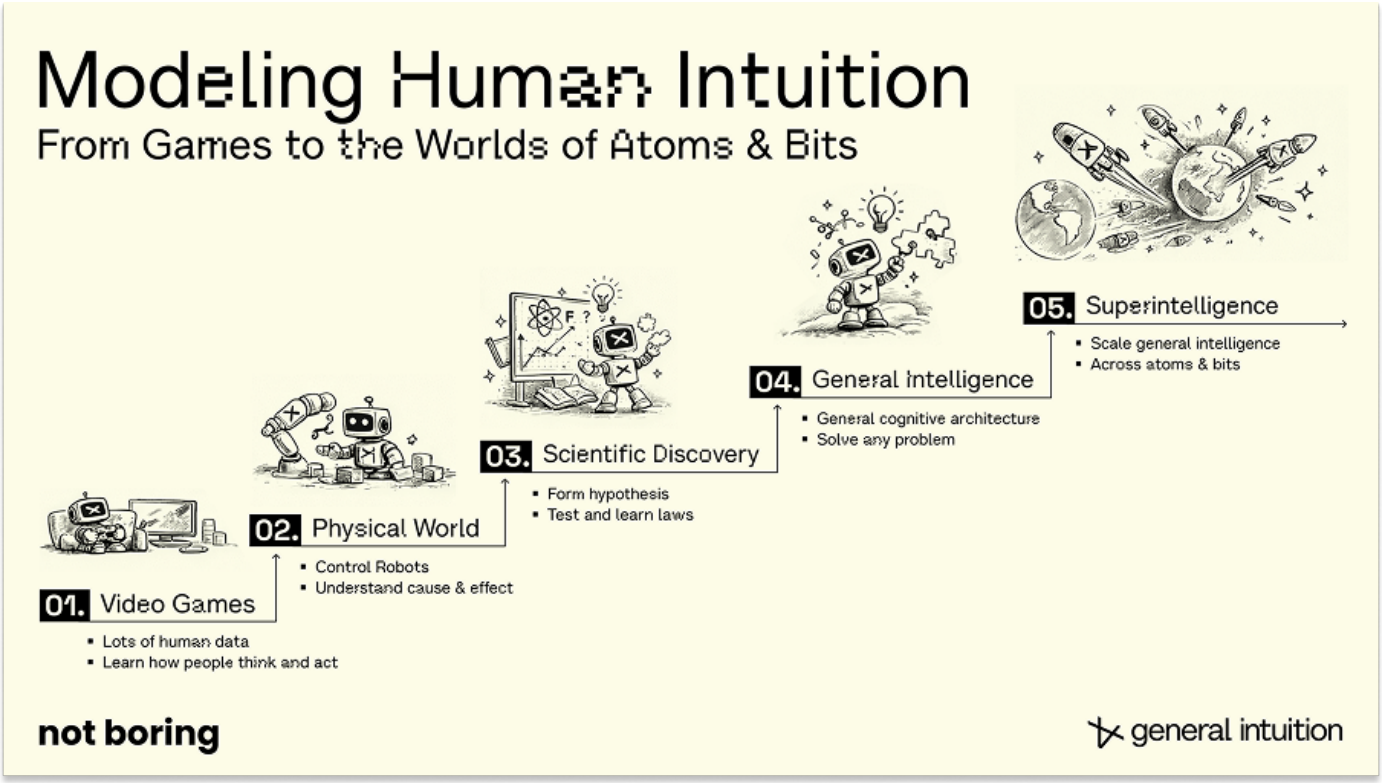
If you get rid of the assumption that our machines don't need to, and probably *shouldn't*, mimic us or take our place in any way, a whole world of possibilities opens up.

At General Intuition, we're actively working on simulations that will eventually allow our systems to go beyond everything that's currently described in pixels, to everything governed by cause and effect. The methods we use are very general. This is a long way out, but a necessary step.

To really understand our world, it turns out, poetically, we may need World Models; compute for the uncomputable.

The implications of all of this are cosmic. If we can model three-dimensionality, physics, and time, and their interaction, then the ability to manipulate these arenas at superhuman macro and micro scales is on the horizon.

There is a tremendous amount of work ahead. Today, nobody is capable of simulating a biological cell, let alone an ecosystem made up of 1030 of them. However, what captivates me is that we don't need to map all of reality's details. We just need to observe how those details manifest in actions, and use those actions to predict what comes next, over and over again.



There is also a tremendous amount of responsibility that comes with building these models, and it's something I take very seriously and personally.

I'm from the generation most at risk of AI displacement; half my childhood friends can't find jobs. I'm spending a lot of time exploring how we bring our community and my generation along in this shift.

For example, like Tesla, Medal sits on over $10 billion of global hardware infrastructure — GPUs, CPUs, plugged into power, with cooling — powered by over 15 million users. We're actively exploring ways to let our community share in what's coming, for example by generating income serving inference from their GPUs, or tele-operating from their gaming rigs. If the demand for general intelligence is anywhere close to what we think, that could be the largest economic tailwind my generation has ever seen.

These are just my dreams for now. But one day, they won't be. One day, we leave the boring problems to superintelligence, so we can explore the stars or the deep sea from our gaming rigs, and dream up the next uniquely human, most interesting, not boring things to do.